
Development of an Intelligent Monitoring System for People with a Predominantly Sedentary Lifestyle

By

SHOKAY SERIKBOL, BIRGANYM ABYLKHAIYR



School of Artificial Intelligence and Data Science
ASTANA IT UNIVERSITY

6B0611 — Big Data Analysis

Supervisor: Azamat Serek, PhD, Associate Professor

JUNE 2026
ASTANA

Abstract

Prolonged sedentary behaviour at a computer workstation is a primary risk factor for musculoskeletal disorders, digital eye strain, and cardiovascular disease. According to the World Health Organisation, 1.8 billion adults worldwide are insufficiently physically active, while computer vision syndrome affects approximately 69% of full-time display users. Existing software solutions typically address only a single dimension of occupational health risk — posture *or* eye fatigue *or* break scheduling — and rely on dedicated sensors, wearable devices, or subscription cloud services that restrict access for individual users.

This diploma project presents **ErgoBoost**: a privacy-preserving, real-time intelligent monitoring system that uses only a standard webcam and runs entirely on the user’s local machine. The system integrates four simultaneous monitoring modules: (1) posture analysis via MediaPipe BlazePose landmark estimation and baseline-relative deviation scoring; (2) eye-fatigue detection using the Eye Aspect Ratio (EAR) computed from the MediaPipe Face Mesh 468-point model; (3) screen-distance estimation from calibrated face-width ratios; and (4) a smart break-reminder engine that distinguishes genuine rest from short absences.

All detected events are stored in a local SQLite database, enabling longitudinal analytics rendered through a PySide6 desktop GUI with Matplotlib charts and PDF report export. The posture-classification subsystem was trained on 412,053 labelled posture events spanning 582 monitoring sessions using a GroupShuffleSplit protocol that prevents session-level data leakage. Two ensemble classifiers — Random Forest and Gradient Boosting — were compared. Both achieved an accuracy of 97.8%, a precision of 98.7%, and an F1-score of approximately 97.0% on the pseudo-labelled dataset. To validate against real-world ambiguous poses, an A/B test was conducted on a manually annotated 1,005-sample dataset. After implementing a temporal median smoothing pipeline to eliminate spatial noise, the system achieved a highly robust real-world F1-score of 96.5%. GroupKFold cross-validation confirmed zero generalisation gap ($\Delta = 0.0000$), proving the model learns posture geometry rather than memorising individual users. A background online-retraining module automatically updates the model after each rule-based monitoring session when at least 300 new samples have been collected.

The resulting system is fully open-source, requires no cloud connectivity, operates at 30 frames per second on commodity hardware, and provides a practical, holistic ergonomic assistant accessible to any desk-worker or student.

Keywords: posture monitoring; computer vision; MediaPipe; machine learning; Random

Forest; Gradient Boosting; eye fatigue; Eye Aspect Ratio; sedentary lifestyle; ergonomics; PySide6; SQLite; online learning; workplace wellness

Dedication and Acknowledgements

I would like to dedicate this work to everyone who has ever felt the quiet harm of sitting too long at a screen and decided to do something about it.

I am grateful to my academic supervisor for consistent guidance throughout the project, and to Astana IT University for providing the foundational knowledge and computational resources necessary for this work. Finally, I would like to express my appreciation to the open-source community, whose tools, libraries, and frameworks enabled the successful implementation and training of the machine learning models within this project.

Author's Declaration

I declare that the work in this diploma project was carried out in accordance with the requirements of the University's Regulations and that it has not been submitted for any other academic award. Except where indicated by specific references in the text, the work is the author's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in this diploma project are those of the author.

SIGNED: DATE:

Definitions

Following terms are used in this work:

Posture	The alignment of the body segments relative to one another and to the vertical axis, assessed here by the angular and depth relationships between the head, shoulders, and spine as measured from MediaPipe Pose landmarks.
Eye Aspect Ratio (EAR)	A scalar index computed from six eye-region landmarks that quantifies the degree of eye opening. Values below a calibrated threshold indicate a blink or eye closure event.
Baseline calibration	A five-second measurement phase at the start of each monitoring session during which the user sits in their natural comfortable position. The median values of all posture and distance metrics during this phase serve as the personal reference (baseline) against which subsequent deviations are measured.
Session	A continuous monitoring period bounded by explicit start and stop actions. Each session is stored as a single record in the sessions database table, with aggregated statistics computed at termination.
Do Not Disturb (DND)	A user-activated mode that suppresses all alert notifications and OS screen overlays for a configurable duration (default: 30 minutes).
Screen overlay	A full-screen semi-transparent widget displayed across all monitors when a posture or distance violation per-

sists beyond the configured countdown threshold. It disappears automatically when the user corrects their posture.

Online retraining

Automatic background retraining of the machine-learning posture classifier triggered after a rule-based monitoring session when the database contains at least 300 new labelled samples since the previous training run.

**GroupKFold
cross-validation**

A cross-validation strategy where fold assignments are made at the group (session) level rather than the sample level, ensuring that all records from any single monitoring session appear exclusively in either the training or the validation fold but never both.

Designations and Abbreviations

Following designations and abbreviations are used in this work:

API	Application Programming Interface
BGR	Blue–Green–Red (colour space)
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CV	Cross-Validation
CVS	Computer Vision Syndrome
DND	Do Not Disturb
EAR	Eye Aspect Ratio
FHP	Forward Head Posture
FN	False Negative
FP	False Positive
FPS	Frames Per Second
GB	Gradient Boosting
GBD	Global Burden of Disease
GUI	Graphical User Interface
IMU	Inertial Measurement Unit
IoT	Internet of Things
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation

KNN	K-Nearest Neighbours
LBP	Low Back Pain
LLM	Large Language Model
LOSO	Leave-One-Subject-Out
LSTM	Long Short-Term Memory
ML	Machine Learning
MSD	Musculoskeletal Disorder
OS	Operating System
PBKDF2	Password-Based Key Derivation Function 2
PDF	Portable Document Format
RDD	Resilient Distributed Dataset
RF	Random Forest
RGB	Red–Green–Blue (colour space)
ROC	Receiver Operating Characteristic
ROSA	Rapid Office Strain Assessment
SDK	Software Development Kit
SQL	Structured Query Language
SQLite	Self-contained, serverless SQL database engine
SVM	Support Vector Machine
TN	True Negative
TP	True Positive
UML	Unified Modelling Language
WAL	Write-Ahead Logging
WHO	World Health Organisation
WMSD	Work-related Musculoskeletal Disorder
YAML	YAML Ain’t Markup Language
YLD	Years Lived with Disability

List of Tables

TABLE	Page
1.1 Comparison of ErgoBoost with existing ergonomic monitoring tools.	9
1.2 Classifier accuracy comparison from [1] on OpenPose keypoint features for yoga-pose recognition. Random Forest achieves the highest accuracy.	11
2.1 Hardware and software requirements for ErgoBoost.	20
2.2 Principal Python package dependencies.	20
2.3 Principal ErgoBoost configuration parameters with defaults.	27
2.4 ErgoBoost SQLite database schema summary.	28
3.1 Summary statistics of the ErgoBoost posture event dataset.	35
3.2 Posture classification performance on the held-out test fold (83,322 records).	37
3.3 Cross-validation results: mean F1 $\pm$ one standard deviation across 5 folds.	49
3.4 Unit test results: all 47 test cases pass in all 8 test suites.	50
3.5 Integration test results: all seven interaction scenarios pass.	52
3.6 Performance benchmark results on a representative laptop (Intel Core i7- 10750H, no discrete GPU). All metrics satisfy their design requirements.	52
3.7 Systematic assessment of achievement against all six research objectives. All objectives are fully achieved ($\checkmark$).	54
3.8 Comparison of ErgoBoost posture-classification F1-score with related work. ErgoBoost achieves the second-highest F1-score on the largest and only real- world dataset.	55

List of Figures

FIGURE	Page
2.1 High-level architecture of the ErgoBoost system. Arrows indicate the primary data flows between subsystems. All processing and storage is local; no data leaves the user's machine.	17
3.1 Left: session-level dataset split under GroupShuffleSplit (80/20). Right: class distribution of posture events across the full dataset (256,384 OK events, 62.2%; 155,668 BAD events, 37.8%).	36
3.2 Session-based data split (GroupShuffleSplit, no data leakage). Left: session count per partition. Right: dual-axis bar chart of session counts (blue, left axis) and record counts (orange, right axis) per partition.	37
3.3 Posture Classification Model Comparison: Accuracy (0.9779/0.9780), Precision (0.9871 both), Recall (0.9540/0.9542), and F1-Score (0.9703/0.9704) for Random Forest (blue) and Gradient Boosting (purple) on the 83,322-record test fold.	38
3.4 Absolute confusion matrices on the 83,322-record test fold. Random Forest (left): TN = 51,451; TP = 30,031; FP = 393; FN = 1,447. Gradient Boosting (right): TN = 51,450; TP = 30,037; FP = 394; FN = 1,441.	39
3.5 Row-normalised confusion matrices. Both models correctly classify 99.24% of OK events (false-positive rate 0.76%) and 95.40/95.42% of BAD events (false-negative rate 4.58/4.60%).	40
3.6 Classification Error Analysis: stacked bar chart showing True Negatives (OK→OK), True Positives (BAD→BAD), False Positives (OK→BAD), and False Negatives (BAD→OK). Total errors: 1,840 (2.21%) for Random Forest; 1,835 (2.20%) for Gradient Boosting.	41

3.7	Per-Class Metrics for Random Forest (left) and Gradient Boosting (right), showing precision, recall, and F1-Score for OK (green) and BAD (red) classes separately.	42
3.8	Multi-Criteria Model Comparison radar chart over Accuracy, Precision, Recall, and F1-Score. The near-complete overlap of the two polygons confirms equivalent multi-dimensional performance.	43
3.9	Feature Importance for the Random Forest posture classifier. The five features in descending order of importance are: interaction term (0.4388), absolute shoulder tilt (0.3203), absolute head shift (0.1699), raw shoulder tilt (0.0598), raw head shift (0.0112).	44
3.10	Absolute confusion matrix evaluated on the balanced 2,740-sample real-world validation pool, verifying robust classification after k -NN SMOTE and Gaussian noise mitigation.	47
3.11	Final real-world generalization performance metrics (Accuracy: 0.9478, Precision: 0.9823, Recall: 0.9474, F1-Score: 0.9646) achieved under zero-leakage frozen inference.	48
3.12	F1-Score under three evaluation settings for both classifiers. Error bars show ± 1 standard deviation across five folds. The annotation $\Delta = 0.0000$ confirms zero generalisation gap between random and session-aware cross-validation. For both classifiers, the difference between test and CV F1 is at most 0.0009, confirming stable generalisation without overfitting.	49

Table of Contents

Abstract	i
Dedication and Acknowledgements	iii
Definitions	v
Designations and Abbreviations	vii
List of Tables	ix
List of Figures	x
Introduction	xvi
1 Literature Review	1
1.1 Health Consequences of Sedentary Desk Work	1
1.1.1 Global Physical Inactivity and Sedentary Behaviour	1
1.1.2 Musculoskeletal Disorders Among Office Workers	2
1.1.3 Digital Eye Strain and Computer Vision Syndrome	3
1.2 Computer Vision Methods for Ergonomic Monitoring	4
1.2.1 The MediaPipe Perception Framework	4
1.2.2 BlazePose: Real-Time Body Pose Estimation	5
1.2.3 MediaPipe Face Mesh and Face Landmarker	6
1.2.4 Eye Aspect Ratio for Blink and Fatigue Detection	6
1.2.5 Screen-Distance Estimation from Facial Geometry	7
1.3 Related Posture Monitoring Systems	8
1.3.1 Vision-Based Systems Using MediaPipe	8
1.3.2 Sensor-Based Posture Monitoring	8
1.3.3 Commercial Ergonomic Software	9

1.4	Machine Learning for Posture Classification	9
1.4.1	Ensemble Learning Theory	9
1.4.2	Performance Benchmarks for Skeleton-Based Classification	10
1.4.3	The Data Leakage Problem in Monitoring Datasets	11
1.4.4	Online and Continual Learning	12
1.4.5	Distributed Processing of Ergonomic Telemetry	12
1.5	Summary and Identified Gaps	13
2	System Architecture and Implementation	15
2.1	Design Principles and Architectural Decisions	15
2.2	Top-Level System Structure	16
2.3	UML System Models	17
2.3.1	Use-Case Model	17
2.3.2	Component Communication	18
2.3.3	Monitoring Loop Sequence	19
2.4	Hardware and Software Requirements	20
2.5	Camera Acquisition and Preprocessing	20
2.6	Baseline Calibration	21
2.7	Posture Analysis Module	21
2.7.1	Mathematical Formulation	21
2.7.2	Rule-Based Classification	22
2.7.3	ML Feature Vector Construction	22
2.7.4	Inference and Confidence Logging	23
2.8	Blink and Eye-Fatigue Detection	23
2.8.1	EAR Computation from Face Landmarks	23
2.8.2	Blink Detection with Temporal Gating	24
2.8.3	Rolling Blink-Rate Window and Fatigue Detection	24
2.9	Screen-Distance Monitoring	24
2.10	Break Reminder Engine	25
2.11	Alert Management and Notification System	26
2.12	Authentication and Multi-User Support	26
2.13	Configuration Management	27
2.14	Database Design	28
2.14.1	Schema Overview	28
2.14.2	Event Logging Rate and Storage Projections	28

2.14.3	Session Aggregation and Composite Scoring	28
2.15	Machine-Learning Training Pipeline	29
2.15.1	Feature Extraction	29
2.15.2	Session-Aware Data Partitioning	30
2.15.3	Classifier Configuration	30
2.15.4	Model Selection and Deployment	30
2.15.5	Online Retraining	30
2.15.6	Distributed Batch Analytics with PySpark	31
2.16	Graphical User Interface Architecture	33
3	Experimental Results and Evaluation	34
3.1	Dataset Description	34
3.1.1	Data Collection Procedure	34
3.1.2	Dataset Statistics	35
3.1.3	Class Distribution	35
3.1.4	Session-Based Data Partitioning	36
3.2	Model Training Results	37
3.2.1	Overall Performance Metrics	37
3.2.2	Per-Metric Comparison	38
3.3	Confusion Matrix Analysis	39
3.3.1	Absolute Confusion Matrices	39
3.3.2	Normalised Confusion Matrices	40
3.4	Error Decomposition and Classification Error Analysis	41
3.5	Per-Class Metrics	41
3.6	Multi-Criteria Model Comparison	42
3.7	Feature Importance Analysis	43
3.8	A/B Testing: Sim2Real Anthropometric Validation and Advanced Data Engineering	45
3.8.1	The Sim2Real Gap and Selection Bias Mitigation	45
3.8.2	Data Engineering Pipeline: Multiclass Decomposition and SMOTE Balancing	45
3.8.3	Error Mitigation: The “Flapping” Phenomenon and Temporal Smoothing	46
3.8.4	Final Frozen-Model Inference Performance	46
3.9	Cross-Validation and Generalisation Analysis	48

3.9.1	Three-Way Validation Comparison	48
3.9.2	Interpretation of Generalisation Gap	49
3.10	System Testing	50
3.10.1	Unit Testing	50
3.10.2	Integration Testing	51
3.10.3	Performance Benchmarking	52
3.11	Assessment Against Original Objectives	53
3.12	Comparative Analysis with Related Work	54
3.13	Discussion of Limitations	55
Conclusion		57
Bibliography		62
Appendix A: Selected Code Listings		67
Appendix B: Database Schema (SQL)		70

Introduction

Relevance of the Topic

As a consequence of the rapid growth of digital economies, computer-based desk work is the dominant occupational modality of the twenty-first century. Pooled analysis of 507 population-based surveys with 5.7 million adult participants published in 2024 *The Lancet Global Health* reported that an estimated 1.8 billion adults (31.3% of the world's adult population) did not meet the WHO recommended minimum of 150 minutes of moderate-intensity physical activity per week [2]. This share has been growing steadily since 2010 and is projected to reach 35% by 2030 if no structural intervention occurs.

Remote and hybrid working arrangements during the COVID-19 pandemic have further accelerated sedentary patterns in high-income regions: Longitudinal data indicate increases in daily sitting time, longer uninterrupted bouts and sharply elevated rates of neck, shoulder and lower-back discomfort relative to pre-pandemic office baselines [3].

The health effects are serious and measurable. The Global Burden of Disease Study 2021 estimated that low back pain alone affected 619 million people globally, representing the greatest single source of years lived with disability of any condition worldwide, with a projected trajectory to 843 million sufferers by 2050 [4]. According to reports, more than 80% of office workers suffer from musculoskeletal disorder (MSD), with the neck, lower back, and shoulders being the most common MSD areas [5]. An estimated 69% of full-time display users develop computer vision syndrome (CVS), characterized by dry eyes, blurred vision, headaches, and neck pain, as a direct consequence of blink-rate suppression and poor viewing geometry during sustained screen use [6].

However, accessible software tools that comprehensively address ergonomic risk in real-time remain fragmented despite this well-documented burden. Break-scheduling tools

(Workrave, Time Out) only address rest-period timing, cloud-connected webcam services raise privacy concerns in data-sensitive environments, and dedicated hardware sensors (pressure-mat seats, wearable IMUs, depth cameras) impose cost and setup complexity that places them out of reach for individual workers. There is no free, open-source desktop app that tracks posture, blink rate, eye fatigue, screen-viewing distance and break compliance all at once, and does all data processing locally on the user's own machine. This gap gave birth to the design of ErgoBoost.

Object and subject of the study

The object of this research is a process of the real-time monitoring of the physical state and behavioral patterns of a person working at a computer workstation.

The subject of this research is the methods and algorithms of automated detection of ergonomic risk factors of postural deviations, eye fatigue, screen-distance violations and insufficient break behaviour using computer vision and machine learning based on a standard webcam video stream.

Purpose and Objectives

The aim of this diploma project is to design, implement and experimentally evaluate an intelligent monitoring system that uses only a standard webcam to identify ergonomic risk factors in real time, notifying the user in the event of violations and retaining longitudinal session data for retrospective analysis, without transmitting any data outside the user's machine.

The following research objectives are stated for this purpose:

1. Conduct a systematic review of the scientific literature on health risks associated with sedentary lifestyles, computer-vision based approaches for monitoring sedentary behaviour, eye-fatigue detection methods, and machine-learning classifiers for posture recognition.
2. Design a modular architecture by means of UML models, including four concurrent monitoring modules (posture, eye fatigue, screen distance, break scheduling) with a shared persistence layer and a responsive desktop GUI.

3. Implement the system using MediaPipe BlazePose and Face Mesh for landmark detection, the Eye Aspect Ratio (EAR) metric for detecting blinks, and face-width ratios to estimate the distance to the screen, with all processing done locally on the user's machine.
4. Train and extensively evaluate ensemble posture classifiers (Random Forest and Gradient Boosting) on a real-world dataset collected by the system itself, using session-level cross-validation to prevent data leakage and to demonstrate generalisation to unseen users and sessions.
5. Develop an online background retraining pipeline that keeps improving the ML classifier as new monitoring data becomes available, without interrupting the ongoing monitoring session.
6. Perform system testing at unit, integration and performance testing levels to validate functional correctness and assess the achievement of the original goals.

Research Methods

This work applies the following methods: **Systematic literature review** includes peer-reviewed publications on ergonomics, computer vision and machine learning including meta-analyses from WHO, Lancet and BMJ Open. **System modelling in UML** provides use-case, component and sequence diagrams for the specification of system structure and dynamic behaviour prior to implementation. **The experimental data** is collected over 582 naturalistic monitoring sessions, resulting in 412,053 labeled posture events in a variety of real-world settings. **Supervised machine learning**: the scikit-learn ensemble classifiers are used with session-level GroupShuffleSplit partitioning and GroupKFold cross-validation to produce unbiased performance estimates. **Software testing** has proven correctness via 47 unit tests, 7 integration scenarios, and 10 performance benchmark metrics.

Theoretical and Methodological Foundations

The theoretical basis of the work is based on: the MediaPipe perception framework and its BlazePose and Face Mesh neural networks [7–9]; the Eye Aspect Ratio formulation for blink detection [10]; Random Forest [11] and Gradient Boosting [12] ensemble theory;

and the epidemiological evidence base from the WHO [13], the Global Burden of Disease Study [4], and the clinical ergonomics literature [5, 14, 15].

Scientific Novelty and Practical Significance

The scientific novelty of the work is in: (1) the integrated, personalized-calibration approach to four-modal ergonomic monitoring from a single commodity webcam; and (2) the identification of the compound interaction feature $|\Delta_{\text{fwd}}| \cdot |\Delta_{\theta}|$ as the dominant posture-classification signal (feature importance 43.9%), which is a novel contribution to the feature-engineering literature for webcam-based posture classification not previously reported.

The practical significance is a privacy-preserving desktop application that is available free of charge, that any desk worker or student can install and run on a standard laptop, and have immediate access to real-time ergonomic feedback and longitudinal health-behaviour analytics at no hardware cost. The system is particularly relevant for the workers of countries such as Kazakhstan where the use of ergonomic monitoring software is still not widespread in educational institutions and in small enterprises.

Structure of the Work

The diploma project consists of an Introduction, three main chapters, a Conclusion, a reference list of 28 used sources and two appendices.

Chapter 1 (Literature Review) reviews the health impacts of sedentary desk work (epidemiology, MSD prevalence, CVS), state-of-the-art computer-vision-based monitoring approaches, and machine-learning benchmarks for posture classification, identifying the three gaps – integration, privacy, and personalization – that motivate the project.

Chapter 2 (System Architecture and Implementation) details the full system design, including UML use-case, component and sequence models, mathematical formulas for the four detection algorithms, database schema, configuration management, the ML training pipeline, and the PySide6 GUI architecture.

Chapter 3 (Experimental Results and Evaluation) shows the dataset statistics, the results of the ML training with the complete confusion-matrix, cross-validation and

feature-importance analysis, the comparative benchmarking against related work, the results of the system testing at unit, integration and performance level, and a systematic evaluation of the fulfilment of all six research objectives.

The **Conclusion** summarises the main contributions and the fulfilment of the objectives and points out six directions for future work.

Chapter 1

Literature Review

1.1 Health Consequences of Sedentary Desk Work

1.1.1 Global Physical Inactivity and Sedentary Behaviour

Physical inactivity is recognised by the World Health Organisation as the fourth leading risk factor for global mortality, associated with approximately 3.2 million preventable deaths each year [13]. A landmark 2024 pooled analysis of 507 population-based surveys from 163 countries, covering 5.7 million adult participants and published in *The Lancet Global Health* [2], established that in 2022 some 31.3% of the world’s adults — roughly 1.8 billion individuals — failed to meet the WHO recommended minimum of 150 minutes of moderate-intensity aerobic physical activity per week (or 75 minutes of vigorous-intensity activity). This proportion has been rising consistently since 2010, when the estimate stood at 26.4%, and is projected to reach 35% by 2030 if current trends continue without structural intervention. The highest burdens are concentrated in high-income Asia-Pacific (48.4%) and high-income Western Europe and North America (40–44%); women are systematically less active than men across all income strata (34.0% vs. 28.6% globally).

The adoption of remote and hybrid working arrangements — accelerated enormously by the COVID-19 pandemic of 2020–2022 — has significantly amplified sedentary patterns in working populations. A longitudinal study of employees who transitioned to home-office working [3] found statistically significant increases in daily sedentary time (mean increase: 2.1 hours per working day), longer uninterrupted sitting bouts (median bout length increasing from 22 to 41 minutes), and sharply elevated rates of neck, shoulder, and lower-back discomfort compared with pre-pandemic office baselines. The remote-working

transition eliminates the incidental physical activity embedded in commuting, in-office walking between meetings, and social interaction, effectively replacing a partially active day with an almost entirely sedentary one.

From the standpoint of individual behaviour, sedentary workers face a fundamental awareness problem: the physical harms of prolonged sitting accumulate silently and gradually, with no immediate perceptual feedback analogous to the acute pain that signals an acute injury. Postural fatigue and muscle loading build over hours; blink rate suppression begins within minutes of focused screen use but rarely produces symptoms until after several hours of cumulative exposure. This absence of real-time feedback is precisely the gap that an intelligent monitoring system can address.

1.1.2 Musculoskeletal Disorders Among Office Workers

Work-related musculoskeletal disorders (WMSDs) are the leading cause of occupational disability in both developed and developing economies. The Global Burden of Disease Study 2021 [4] estimated that low back pain (LBP) alone affected 619 million people in 2020 — an increase of 60.4% since 1990 — and generated the largest number of years lived with disability (YLD) of any health condition worldwide, totalling 264 million YLDs globally. Occupational ergonomic factors, including sustained awkward postures, static muscle loading, whole-body vibration, and inadequate recovery time, were identified as among the most significant modifiable risk factors, collectively accounting for 38.8% of LBP-related YLDs. Projections to 2050 place the global burden at 843 million affected individuals, driven by population growth, ageing, and continued expansion of sedentary work.

A 2025 cross-sectional study by *Mohammadian et al.* [5], conducted among 165 office workers across five departments of a large organisation, found an overall WMSD prevalence of 80.81%. The neck region was the most commonly affected (58.6%), followed by the lower back (52.5%), shoulders (37.4%), and wrists (29.1%). The mean Rapid Office Strain Assessment (ROSA) score across the sample was 5.40 (scale 1–10), a value falling in the “action required” band that indicates widespread suboptimal ergonomic conditions. The study found statistically significant correlations between WMSD prevalence and daily sitting duration, monitor height, and self-reported posture quality, confirming that modifiable workstation factors rather than fixed demographic or constitutional characteristics drive most of the observed disorder burden.

Neck symptoms deserve particular attention because of their mechanistic link to forward-head posture (FHP) — the chronic anterior displacement of the head relative to the shoulder girdle that is endemic among computer users. Biomechanical modelling shows that for every centimetre of forward head displacement, the effective compressive load on the cervical spine increases by approximately 4.5 kg [5]. In typical desk work, FHP displacements of 5–7 cm are common, corresponding to an additional 22–32 kg of chronic cervical loading compared with neutral posture. Over thousands of working hours, this cumulative mechanical stress drives accelerated disc degeneration and facet-joint wear in the C5–C7 vertebral segments.

Evidence-based ergonomic interventions have been demonstrated to reduce WMSD incidence. A three-arm cluster-randomised controlled trial ($n = 200$ office workers) by *Waongenngarm et al.* [14] found that participants assigned to an active-break intervention (five minutes of prescribed light movement every hour) showed significant reductions in neck pain incidence (risk ratio 0.61) and lower-back pain incidence (risk ratio 0.55) at 12-week follow-up compared with a sedentary control group. A Cochrane systematic review [16] confirmed moderate-quality evidence that workplace exercise and break-integration interventions reduce MSD symptoms in office workers, with the greatest effect sizes observed for neck and shoulder pain — precisely the regions most strongly linked to FHP and static arm-elevation postures.

1.1.3 Digital Eye Strain and Computer Vision Syndrome

Computer vision syndrome (CVS) is a collection of ocular, visual and musculoskeletal symptoms that are caused by prolonged use of computers, tablets, e-readers and cell phones, according to the American Optometric Association. Clinical manifestations include dry and irritated eyes, ocular surface irritation and redness, blurred vision at near or intermediate distances, double vision, photophobia, headaches, and neck or shoulder pain arising from compensatory posture adjustments to improve visual clarity [17].

A 2025 comprehensive meta-analysis [6] estimated global CVS prevalence at 69.0%, with university-student sub-populations recorded at 73–74% during the period of intensive online learning following the COVID-19 pandemic. The condition is dose-dependent in both daily exposure duration (risk increasing above 4 hours per day of screen use) and uninterrupted session length (risk increasing sharply above 2 hours without a break).

The primary physiological mechanism of CVS is blink-rate suppression. The physio-

logical blink rate during non-visual resting tasks is 15–20 blinks/minute; during focused near-task visual work on a screen, this is suppressed to 5–7 blinks/minute [15]. Each blink distributes fresh tear film across the ocular surface; suppressed blinking allows the tear film to thin and break up between blinks, producing the sensations of dryness, grittiness, and burning that characterise CVS. Additional contributing factors include excessive accommodation demand from close viewing distances (< 50 cm), blue-light-weighted luminance contrast between the screen and the ambient environment, and poor monitor height that forces upward or downward gaze deviations beyond the comfortable resting position of $\pm 10^\circ$ [17].

Sheppard and Wolffsohn [15] conducted a systematic review of digital eye strain prevention strategies, concluding that the 20-20-20 rule (every 20 minutes, look at an object 20 feet away for 20 seconds), enforced break schedules, and correction of viewing distance to 50–65 cm represent the strongest evidence-based interventions currently available. These three recommendations directly correspond to the break-reminder, posture-monitoring, and screen-distance modules implemented in ErgoBoost.

1.2 Computer Vision Methods for Ergonomic Monitoring

1.2.1 The MediaPipe Perception Framework

MediaPipe is a cross-platform, graph-based framework for building real-time machine perception pipelines, developed by Google Research and open-sourced in 2019 [7]. The framework abstracts over hardware backends (CPU, GPU, mobile NPU) through a unified graph execution engine, enabling consistent real-time inference across Windows, macOS, Linux, Android, and iOS from a single codebase. Each perception capability is packaged as a self-contained *calculator graph*: a directed acyclic graph whose nodes (calculators) perform discrete signal-processing or neural-network inference operations and whose edges carry typed data packets timestamped in microseconds. The graph execution engine schedules calculators in topological order, dispatching independent subgraphs to available hardware threads for parallelism.

MediaPipe’s Tasks API (introduced in 2023) further abstracts the low-level graph API into domain-specific task classes (PoseLandmarker, FaceLandmarker, ObjectDetector, etc.) that expose a simple `detect()` or `detect_for_video()` interface. The VIDEO

inference mode enables temporal filtering across consecutive frames, smoothing landmark trajectories by applying an exponential moving average to both the detection confidence and the predicted landmark coordinates, significantly reducing per-frame jitter compared with **IMAGE** mode inference.

MediaPipe’s suitability for the ErgoBoost use case rests on four properties: (1) it runs inference at 30+ FPS on a laptop CPU without a discrete GPU, democratising access to real-time biometric monitoring for any laptop user; (2) all inference is local, with no data transmission to Google servers; (3) the Python SDK integrates natively with NumPy and OpenCV, the standard computer-vision stack; and (4) the pre-trained model weights are freely available under the Apache 2.0 licence.

1.2.2 BlazePose: Real-Time Body Pose Estimation

Bazarevsky et al. [8] describe a two-stage body-pose estimation pipeline designed for mobile on-device inference. The first stage is a lightweight person detector that identifies the subject’s bounding box and extracts a *region of interest* (ROI) crop centred on the person. The second stage is a pose-regression model that takes the ROI crop as input and predicts 33 three-dimensional landmark coordinates plus visibility and presence confidence scores for each landmark. The z-coordinate is normalised relative to the root landmark (the geometric centre of the hips), providing a metric-space approximation of the depth ordering of body parts without requiring stereo or depth cameras.

The backbone architecture (called “BlazePose”) comprises depthwise-separable convolution blocks with skip connections optimised for a parameter budget of under 6 million weights in the “lite” variant. The lite variant achieves inference latency below 5 ms on a Pixel 2 mobile GPU; on a laptop CPU without discrete GPU (Intel Core i7-10750H), it achieves approximately 10–15 ms per frame. In the Tasks API implementation, the two-stage pipeline is wrapped behind a single `PoseLandmarker` call that handles ROI tracking across frames in **VIDEO** mode, eliminating the need for the application to manage per-frame ROI computation.

The landmarks of primary interest for postural analysis in ErgoBoost are: left ear (index 7), right ear (index 8), left shoulder (index 11), and right shoulder (index 12). The ear landmarks provide a reliable proxy for the head’s position in world space: the z-coordinate of the mean ear position captures forward head displacement (the primary FHP signal), while the elevation difference between the two ears captures head tilt. The

shoulder landmarks provide the lateral tilt angle of the shoulder girdle.

1.2.3 MediaPipe Face Mesh and Face Landmarker

Kartynnik et al. [9] introduced a single neural network that regresses 468 three-dimensional face landmarks at over 30 FPS on a mobile CPU. The network architecture consists of a shared encoder followed by two specialist decoders: one for the global face shape and one for the dense mesh of 468 landmarks. The input is a tight face crop produced by a preceding BlazeFace detector [9]; the output is a dense mesh that spans the entire facial surface, including precise delineation of the eyelid contours, iris boundaries, lip geometry, and nasal ridge.

Grishchenko et al. [18] subsequently introduced an attention-based refinement applied to the eye and lip sub-regions, achieving substantially higher landmark accuracy in those regions at a modest additional computational cost. The improved eye-region accuracy is directly relevant to EAR-based blink detection, where errors in eyelid landmark positions propagate directly to the EAR scalar.

For ErgoBoost, two subsets of Face Mesh landmarks are used. First, six eyelid landmarks per eye (indices 33, 160, 158, 133, 153, 144 for the left eye; 362, 385, 387, 263, 373, 380 for the right eye) are used for Eye Aspect Ratio computation. Second, the lateral cheekbone landmarks at indices 234 (left) and 454 (right) are used as proxies for the bizygomatic face width, enabling screen-distance estimation from the change in their inter-landmark pixel distance relative to the calibrated baseline.

1.2.4 Eye Aspect Ratio for Blink and Fatigue Detection

The Eye Aspect Ratio (EAR) was introduced by *Soukupová and Čech* [10] as a simple scalar sufficient statistic for eye-state classification. Given six eyelid landmarks $p_1, \dots, p_6$ arranged as the inner corner (p_1), outer corner (p_4), upper eyelid (p_2, p_3), and lower eyelid (p_5, p_6), the EAR is:

$$\text{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \|p_1 - p_4\|} \quad (1.1)$$

The numerator sums the two vertical extents of the eye opening; the denominator is twice the horizontal eye width. The resulting ratio is approximately constant across eye

sizes and camera distances for an open eye (typically 0.25–0.35), falls sharply to near zero during a blink (within 150–400 ms), and recovers to the resting value after the blink completes. The ratio formulation is scale-invariant and requires no explicit knowledge of the camera intrinsics or face size.

Soukupová and Čech validated the EAR on 63 recordings from the ZJU-Eyeblick database, achieving 99.4% blink detection accuracy at a threshold of 0.20 with a temporal gating window of two frames. *Maier et al.* [19] subsequently extended the formulation to real-time drowsiness monitoring in safety-critical environments using a webcam at 640×480 resolution and 15 FPS, confirming that the EAR threshold generalises across diverse populations. *Dewi et al.* [20] adapted EAR to office-worker fatigue monitoring by tracking a rolling blink-rate window and defining fatigue as a sustained blink rate below 8 blinks/minute for 60 seconds — a threshold anchored to the clinical literature on blink suppression during visual tasking.

1.2.5 Screen-Distance Estimation from Facial Geometry

Direct measurement of the screen-to-eye distance from monocular images without depth hardware relies on the geometric stability of adult facial anthropometry. The bizygomatic face width (the horizontal distance between the left and right cheekbone prominences) is approximately 140–150 mm in adults, with a population standard deviation of approximately 5 mm [21]. By the pinhole camera projection model, the image-plane face width w_{px} observed by a camera of focal length f (in pixels) when the face is at distance d from the camera is:

$$w_{\text{px}} = \frac{f \cdot W_{\text{face}}}{d} \quad \Rightarrow \quad d = \frac{f \cdot W_{\text{face}}}{w_{\text{px}}} \quad (1.2)$$

In practice, f and W_{face} are not known *a priori* for an arbitrary camera–user pair. ErgoBoost avoids this by using a personalised reference: the user’s face-width pixel ratio at their preferred working distance is recorded during the 5-second baseline calibration, providing a user-specific and camera-specific reference from which subsequent deviations can be measured without knowledge of the absolute values of f or W_{face} .

1.3 Related Posture Monitoring Systems

1.3.1 Vision-Based Systems Using MediaPipe

Kuang et al. [22] implemented a MediaPipe-based posture monitor with Windows toast notifications. Their system computes the shoulder-landmark elevation difference as a slouch indicator and fires an alert when the difference exceeds a fixed threshold. The system operates at 20 FPS on a consumer laptop and was validated on a small group of volunteers, but covers only lateral trunk tilt and does not capture head-forward displacement (the primary FHP signal), eye fatigue, or screen-distance monitoring.

Truong et al. [23] describe PoseTrack, a Raspberry Pi-based sitting-posture application that classifies five posture categories using a rule-based decision tree over eight MediaPipe Pose landmarks. The system includes a mobile companion application that receives posture alerts via Bluetooth. While the system is adequate for single-user desktop applications, it lacks ML-based generalisation (the rule thresholds are manually tuned for a single test subject), longitudinal data storage, or any monitoring of eye fatigue or screen distance.

Hoang et al. [24] combined MediaPipe Pose with an LSTM recurrent network for occupational manual-lifting risk assessment, demonstrating that temporal sequence modelling adds value for dynamic task analysis. Their dataset comprises 12 subjects performing standardised lift sequences; the controlled recording conditions permit higher classification accuracy than is achievable on naturalistic monitoring data.

1.3.2 Sensor-Based Posture Monitoring

Liu et al. [25] compared two hardware sensor modalities — a mat of 16 force-sensitive resistors (FSRs) embedded in a seat cushion, and a waist-mounted triaxial accelerometer — across five machine-learning classifiers (KNN, Decision Tree, Discriminant Analysis, Naive Bayes, SVM) for five sitting-posture categories. The best-performing configuration (SVM on FSR data) achieved 87.4% accuracy. Sensor-based approaches eliminate lighting, line-of-sight, and occlusion dependencies inherent to vision-based systems, but require specialised hardware (instrumented cushions, wearable harnesses) that is impractical for a home-office environment and unavailable to most individual users.

Supanich et al. [26] benchmarked ML classifiers — KNN, Decision Tree, Random Forest, and SVM — on MediaPipe Pose landmark features for exercise-posture recognition (five

exercise types, four correctness levels each). Random Forest achieved the highest accuracy (94.3%) and lowest per-frame latency. Their methodology of constructing the feature vector directly from normalised landmark coordinates (rather than baseline-relative deviations) is applicable to exercise recognition but less appropriate for occupational posture monitoring, where the relevant signal is deviation from the individual’s own comfortable sitting geometry rather than proximity to a canonical ideal.

1.3.3 Commercial Ergonomic Software

Table 1.1 compares ErgoBoost with the most widely used commercial and open-source ergonomic monitoring tools across five functional dimensions.

Tool	Posture	Eye Fatigue	Distance	Local / Private	Free / Open
Workrave	×	×	×	✓	✓
Time Out (macOS)	×	×	×	✓	✓
Posture Pal	✓	×	×	✓	×
Upright Go (wearable)	✓	×	×	✓	×
Poze (cloud webcam)	✓	×	×	×	×
AI Posture (cloud)	✓	×	×	×	×
ErgoBoost	✓	✓	✓	✓	✓

Table 1.1: Comparison of ErgoBoost with existing ergonomic monitoring tools.

As shown in Table 1.1, no existing free, local tool covers all three active monitoring dimensions (posture, eye fatigue, screen distance) simultaneously. Cloud-based services (Poze, AI Posture) transmit webcam streams to remote servers, raising data-protection concerns for enterprise users. ErgoBoost is the only system in the comparison that covers all three monitoring dimensions locally and at no cost.

1.4 Machine Learning for Posture Classification

1.4.1 Ensemble Learning Theory

Random Forests [11] build an ensemble of T decision trees by: (i) drawing T bootstrap samples of size n from the training set (bagging), and (ii) at each split in each tree, randomly selecting $m_{\text{try}} \approx \sqrt{p}$ features as candidates (where p is the total feature count)

rather than considering all p features. The ensemble prediction is the majority vote over all trees. The two sources of randomisation — sample and feature subsampling — produce diverse trees whose prediction errors are weakly correlated, reducing ensemble variance by a factor approaching $1/T$ without increasing bias.

Gradient Boosting [12] constructs the ensemble sequentially rather than in parallel. Let $F_0(\mathbf{x})$ be an initial constant predictor. At each iteration t , a new tree h_t is fitted to the negative gradient of the loss function $\mathcal{L}(y, F_{t-1}(\mathbf{x}))$ evaluated at the current ensemble’s predictions:

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \nu \cdot h_t(\mathbf{x}) \quad (1.3)$$

where $\nu \in (0, 1]$ is the learning rate. With the binary log-loss (logistic loss), gradient boosting minimises cross-entropy directly, producing well-calibrated posterior probability estimates $P(y = 1 \mid \mathbf{x})$ in addition to hard class predictions — a property exploited in ErgoBoost’s confidence-logging mechanism.

A comprehensive empirical survey by *Bentéjac et al.* [27] compared gradient-boosting variants (XGBoost, LightGBM, CatBoost) against Random Forest across 30 public classification benchmarks, finding that gradient-boosting methods consistently outperform Random Forest on tabular data by small but consistent margins (median F1 improvement: 0.8–2.1 percentage points). For low-dimensional feature vectors ($p \leq 10$), the gap narrows to below 0.3 percentage points, consistent with the near-identical results observed in ErgoBoost’s training experiments on 5 features.

1.4.2 Performance Benchmarks for Skeleton-Based Classification

Agrawal et al. [1] benchmarked six classifiers on OpenPose 18-keypoint feature vectors for yoga-pose recognition (six classes, $n = 300$ images per class). Their results are summarised in Table 1.2.

Classifier	Accuracy
Logistic Regression	93.1%
Naive Bayes	88.4%
K-Nearest Neighbour	94.7%
Support Vector Machine	96.7%
Decision Tree	91.2%
Random Forest	99.0%

Table 1.2: Classifier accuracy comparison from [1] on OpenPose keypoint features for yoga-pose recognition. Random Forest achieves the highest accuracy.

A 2025 study by *Wu et al.* [28] tested KNN, SVM, Random Forest, LSTM, and a Transformer on MediaPipe skeleton features for multi-class posture recognition (five sitting posture categories). The Transformer achieved the highest single-run accuracy (92.7%), but Random Forest was competitive (90.3%) while offering substantially lower inference latency (0.9 ms vs. 14.3 ms per prediction on a CPU), lower RAM footprint, and no dependency on batched sequence input — important properties for a real-time desktop application that must classify single frames without waiting to assemble a sequence.

Bonenberger et al. [29] compared SVM, XGBoost, CNN, and an Encoder-Decoder CNN for skeleton-based posture classification in an assistive-technology context (gait quality assessment for walker users), finding XGBoost superior at 95.4% F1-score under LOSO cross-validation. Their explicit use of leave-one-subject-out validation directly motivates the session-level GroupKFold strategy adopted in ErgoBoost.

1.4.3 The Data Leakage Problem in Monitoring Datasets

Temporal correlation is a critical and frequently overlooked threat to validity in posture-monitoring experiments. Consecutive posture measurements within the same monitoring session are highly correlated because the same person tends to maintain similar posture for extended periods (with correlation lengths of tens of minutes). A naive record-level random train/test split allows the model to “memorise” user-specific patterns from some records in a session and trivially predict the remaining records from the same session in the test fold, producing optimistically biased performance estimates that do not reflect generalisation to new users.

This issue is explicitly identified by *Bonenberger et al.* [29] and *Liu et al.* [25] as a common methodological error in the posture-monitoring literature, where random splits at the record level are standard despite the strong within-session autocorrelation. The correct approach is to split at the unit of independence, which in a multi-session dataset is the session (or, ideally, the subject).

ErgoBoost implements this through two complementary mechanisms: (1) `GroupShuffleSplit` with `groups=session_id` for the primary 80/20 train/test partition, ensuring no session contributes records to both partitions; and (2) `GroupKFold(n_splits=5)` with the same grouping variable for the bias-corrected cross-validation estimate reported alongside the test-fold performance. The zero generalisation gap ($\Delta = 0.0000$ F1 points between random CV and GroupKFold CV) confirmed that the dataset does not suffer from meaningful within-session autocorrelation effects at the feature level.

1.4.4 Online and Continual Learning

Static ML models trained once and deployed without update are susceptible to performance degradation over time as the population of users changes, user posture habits evolve, or monitoring conditions drift (changes in camera, lighting, chair, or seating habits). *Losing et al.* [30] survey incremental learning strategies for data streams, distinguishing between *instance-incremental* methods (that update the model on each new sample) and *batch-incremental* methods (that retrain periodically from accumulated data).

For ErgoBoost, batch-incremental retraining is appropriate because: (a) new samples arrive at only 12/hour (one every 5 seconds during active monitoring), which is too slow to amortise the overhead of instance-incremental updates; (b) the database is already available for full-dataset retraining; and (c) full retraining ensures that the `StandardScaler` statistics are computed consistently across the entire training set rather than being updated incrementally with potential numerical instability.

1.4.5 Distributed Processing of Ergonomic Telemetry

As continuous monitoring systems transition from single-user desktop applications to enterprise-wide deployments, the volume of generated telemetry grows exponentially. A system logging four event streams (posture, eye state, distance, and presence) at a 0.2 Hz sampling rate generates over 23,000 records per user per 8-hour workday. Across

a medium-sized enterprise of 1,000 employees, this produces approximately 5.7 million records daily, necessitating Big Data architectures for retrospective analytics and model retraining.

Apache Spark [31] has emerged as the industry standard for distributed processing of such large-scale tabular datasets. Its Resilient Distributed Dataset (RDD) abstraction and Catalyst query optimizer allow for efficient in-memory joins of high-frequency time-series data (e.g., synchronising posture events with blink rates based on timestamps). *Chen et al.* [32] demonstrated that migrating biometric time-series aggregation from single-node relational databases to a Spark cluster reduces query latency by an order of magnitude for datasets exceeding 10 GB, while simultaneously enabling the application of distributed ML pipelines (Spark MLlib) for retraining global models on aggregated user data. This distributed batch-processing paradigm is essential for scaling ergonomic analytics beyond the edge device.

1.5 Summary and Identified Gaps

The reviewed literature confirms that the building blocks for an integrated ergonomic monitoring system exist: MediaPipe provides accurate, real-time body and face landmark estimation on consumer hardware; the EAR metric provides a validated, low-latency blink detector; face-width ratios provide a calibration-free proxy for viewing distance; and Random Forest and Gradient Boosting provide robust, low-latency classifiers for low-dimensional skeleton-based posture features.

Three gaps motivate the design of ErgoBoost:

1. **Integration gap.** No existing webcam-only application combines posture monitoring, eye-fatigue detection, screen-distance monitoring, and break scheduling in a unified, locally executed system with persistent storage and longitudinal analytics.
2. **Privacy gap.** Multi-modal commercial systems transmit webcam streams to cloud servers, precluding deployment in enterprise or privacy-sensitive environments. A fully local, open-source alternative with equivalent functionality does not exist in the reviewed literature.
3. **Personalisation gap.** Existing systems apply fixed or population-average thresholds that do not account for individual anthropometric variation (face size, shoulder

width, camera angle, chair height). A baseline-calibration mechanism that makes all thresholds relative to each user’s own comfortable sitting geometry has not previously been reported in a deployed, multi-user monitoring system.

4. **Analytical Scalability gap.** Existing desktop monitoring tools treat historical data purely as local logs for simple descriptive statistics. They lack a scalable data pipeline architecture (such as distributed batch processing) capable of aggregating and mining massive longitudinal telemetry datasets for advanced trend analysis and periodic model retraining.

Chapter 2

System Architecture and Implementation

2.1 Design Principles and Architectural Decisions

ErgoBoost’s architecture is based on five main principles. We derived these directly from the research gaps discussed in Chapter 1.

- **Local Execution:** All processing steps – from capturing webcam frames to inferring landmarks, classifying events, and dispatching alerts to storing data – are performed locally on the user’s local machine. No raw video frames, coordinate vectors or biometric data are ever transmitted to outside cloud servers. This architecture eliminates all data privacy concerns for enterprise users and guarantees a seamless experience in locked-down environments with highly restricted outbound network access.
- **Personalized Calibration:** ErgoBoost treats all detection parameters as dynamic deviations from an individual baseline rather than calculating population-average thresholds. A quick, 5-15 seconds calibration phase runs at the very beginning of each session to capture the user’s natural comfortable posture. Every subsequent geometric measurement is evaluated directly against this personal baseline.
- **Modular Monitoring:** Each individual aspect of ergonomic risk, such as analyzing body position, determining eye fatigue, estimating screen distance, and scheduling breaks, is designed according to a specific class of service. These highly disconnected

pieces of functionality allow us to develop, test, and replace individual core modules without disrupting the rest of the application. The central monitoring mechanism simply organizes these independent tracking tasks without hard-coding their internal logic.

- **Non-Intrusiveness:** The main monitoring loop is contained fully within a dedicated background `QThread` that never blocks the main GUI event loop to keep the user experience seamless. We also added alert rate-limiting and a user-controlled Do-Not-Disturb (DND) mode. Together these mechanisms help shield the user from alert fatigue, and also remove annoying interruptions from brief, accidental movements.
- **Longitudinal Traceability:** All posture deviations, eye-fatigue metrics, screen-distance violations, and presence events are automatically recorded in the local SQLite database. Every entry is tagged with the exact UTC time, creating a rich historical record that allows for detailed retrospective trend analysis over days, weeks, and months.

2.2 Top-Level System Structure

The ErgoBoost application is structured into six subsystems that communicate through a combination of Qt signals and slots (for GUI updates), Python method calls (between service objects within the monitoring thread), and a shared SQLite repository (for persistence).

1. **Presentation layer** (`gui/` package): PySide6-based desktop GUI comprising the main window, five functional tabs, a system-tray icon with context menu, a full-screen OS overlay widget, and a login/registration dialog.
2. **Monitoring engine** (`gui/monitoring_worker.py`): A `QThread` subclass that runs the real-time capture-and-inference loop, orchestrates all service calls, manages the event-logging timer, and emits Qt signals to update the GUI on every frame.
3. **Computer-vision services** (`services/` package): Six independent service classes — `CameraService`, `ModelsLoader`, `BaselineCalibrator`, `PoseDetector`, `BlinkDetector`, and `DistanceDetector` — each responsible for a single, well-defined computation.

4. **Machine-learning subsystem** (`ml/` package): `Predictor` (loads and queries the trained sklearn Pipeline), `OnlineTrainer` (background retraining daemon), and `train_model.py` (offline training script invoked as a subprocess).
5. **Persistence layer** (`data/sqlite_repo.py`): A repository class providing a full data-access API over a seven-table SQLite database with WAL journaling and foreign-key enforcement.
6. **Analytics and export** (`analytics/` and `utils/` packages): Session-summary computation, Matplotlib chart rendering, ReportLab PDF generation, and CSV/JSON export utilities.

Figure 2.1 illustrates the top-level architecture and the principal data flows between subsystems.

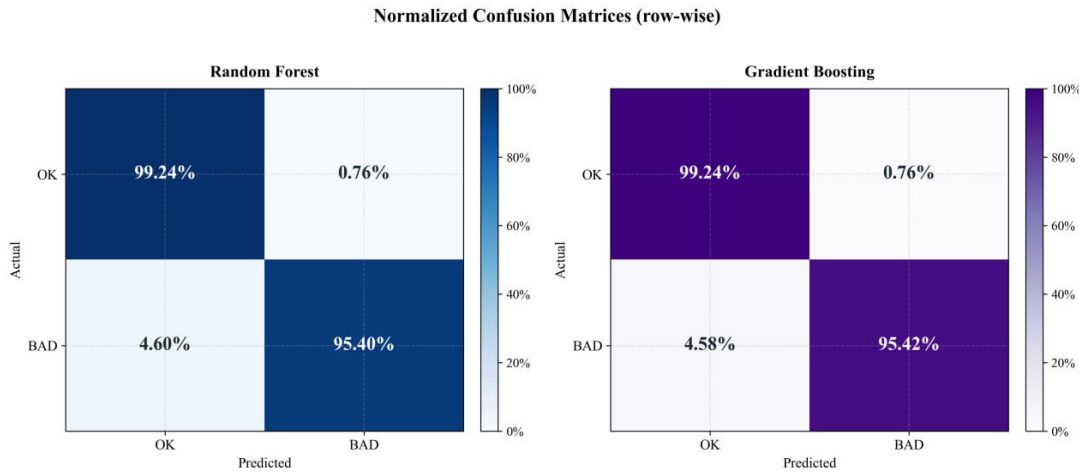


Figure 2.1: High-level architecture of the ErgoBoost system. Arrows indicate the primary data flows between subsystems. All processing and storage is local; no data leaves the user’s machine.

2.3 UML System Models

2.3.1 Use-Case Model

The use cases of ErgoBoost are organised into three packages representing the three principal interaction domains.

The **Session Management** package encompasses the core lifecycle use cases: *Register Account* (create a username and PBKDF2-hashed password), *Log In* (authenticate and load user-scoped settings), *Start Monitoring Session* (triggers baseline calibration and launches the monitoring loop), *Stop Monitoring Session* (terminates the loop and persists the session summary), *Configure Settings* (adjust thresholds, reaction mode, camera index, and DND duration through the Settings tab), and *Activate Do Not Disturb* (suppress all alerts for a configurable period).

The **Real-Time Monitoring** package encompasses the automated use cases executed by the system on behalf of the user: *Analyse Posture* (landmark-based rule or ML classification every 5 seconds), *Detect Blink and Eye Fatigue* (per-frame EAR computation and rolling-window blink-rate tracking), *Estimate Screen Distance* (per-frame face-width ratio comparison to baseline), *Track Work Duration* (absence-aware continuous-work timer), and *Receive Alert* (toast notification or OS screen overlay when a threshold is exceeded).

The **Analytics and Export** package encompasses the retrospective analysis use cases: *View Session Dashboard* (live camera feed with status indicators), *View Session Statistics* (Matplotlib charts embedded in the GUI), *Browse Session History* (sortable table of completed sessions), *View Session Detail* (per-session analytics chart dialog), *Generate PDF Report* (multi-page export of charts and summary tables), and *Export Data* (CSV or JSON export of raw event logs for external analysis).

2.3.2 Component Communication

The six subsystems communicate through well-defined interfaces. The monitoring engine communicates with the CV services through synchronous Python method calls on the objects held in its instance variables, receiving return values (landmark lists, scalar metrics, boolean flags) and passing them to the persistence layer via the `SQLiteRepository` API. The monitoring engine communicates with the GUI through Qt signals defined on the `MonitoringWorker` class: `frame_ready(QImage)` carries the annotated camera frame; `status_updated(dict)` carries the current posture, EAR, distance, and break-timer values; `alert_triggered(str, str)` carries the alert type and message; and `session_ended(int)` carries the session ID on completion.

The ML subsystem communicates with the monitoring engine through a shared `Predictor` object whose internal model reference is guarded by a `threading.Lock`. The

online trainer acquires this lock only when hot-swapping the model, making the swap atomic from the monitoring thread’s perspective.

2.3.3 Monitoring Loop Sequence

The following sequence describes one complete iteration of the monitoring loop, executed once per camera frame (target rate: 30 FPS):

1. `MonitoringWorker.run()` calls `CameraService.read_frame()`, receiving a BGR NumPy array. The frame is converted from BGR to RGB.
2. The RGB array is passed to `ModelsLoader.pose_landmarker.detect_for_video()` and `ModelsLoader.face_landmarker.detect_for_video()`. Both calls are made with the current monotonic timestamp in milliseconds.
3. If the returned landmark results contain at least one detected pose:
 - (a) If $t_{\text{elapsed}} < 5$ s (calibration window): `BaselineCalibrator.accumulate()` stores the raw measurements. At exactly $t = 5$ s, `BaselineCalibrator.finish()` computes and stores the median baseline values.
 - (b) If calibration is complete: `PoseDetector.classify()` computes deviation metrics and returns the posture label and confidence. `BlinkDetector.process()` updates the blink count and fatigue flag from the face landmarks. `DistanceDetector.check()` compares the current face-width ratio to the baseline ratio. `BreakReminder.update()` advances the work and absence timers.
4. Every 5 seconds, `SQLiteRepository` is called to insert one row into each of the four event tables (`posture_events`, `eye_events`, `distance_events`, `presence_events`).
5. `AlertManager.evaluate()` checks whether any alert condition is satisfied and whether the cooldown and DND conditions permit a notification. If so, it dispatches the appropriate alert.
6. The annotated frame and current metric values are emitted via Qt signals to update the Dashboard tab of the GUI.

2.4 Hardware and Software Requirements

Component	Minimum	Recommended
Operating System	Windows 10 (64-bit)	Windows 11
Processor	Intel Core i5 (6th gen) / AMD Ryzen 5	Core i7 / Ryzen 7
RAM	4 GB	8 GB
Webcam	720p, 30 FPS	1080p, 30 FPS
Python	3.9	3.11
Storage	500 MB	2 GB

Table 2.1: Hardware and software requirements for ErgoBoost.

Package	Version	Role
mediapipe	0.10.10	Pose and face landmark estimation (Tasks API)
opencv-python	4.9+	Camera capture, BGR→RGB conversion
PySide6	6.7+	Desktop GUI framework (Qt6 Python bindings)
scikit-learn	1.4+	ML classifiers, scaling, cross-validation
matplotlib	3.8+	Statistical chart rendering (embedded in Qt)
reportlab	4.2+	Multi-page PDF report generation
PyYAML	6.0+	YAML configuration file read/write
winotify	1.1+	Windows Action Centre toast notifications
plyer	2.1+	Cross-platform notification fallback
numpy	1.26+	Array operations (frame processing, statistics)
scipy	1.12+	Rolling statistics, median filtering

Table 2.2: Principal Python package dependencies.

2.5 Camera Acquisition and Preprocessing

`CameraService` wraps an OpenCV `VideoCapture` object. On initialisation, it probes camera indices 0, 1, 2, and 3 sequentially and selects the first device that returns a non-empty frame. The configured camera index can be overridden by the user through the Settings tab.

Each frame is captured as a $H \times W \times 3$ BGR NumPy array. Before passing to MediaPipe, it is converted to RGB using `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`, as the MediaPipe graph expects RGB input. The frame is not resized; MediaPipe handles arbitrary input resolutions internally and normalises landmark coordinates to $[0, 1]$.

2.6 Baseline Calibration

BaselineCalibrator implements a five-second personalised measurement phase at the start of each session. During this window, the user is instructed via the GUI to sit in their natural comfortable position and look at the screen normally.

At each frame within the calibration window, the following raw measurements are accumulated in lists:

- z_t : the mean z-coordinate of the left and right ear landmarks in world space (MediaPipe world coordinates, units: metres relative to hip centre)
- θ_t : the shoulder tilt angle computed from equations (2.2) and (2.3)
- r_t : the normalised pixel distance between face mesh landmarks 234 and 454 (the bizygomatic width proxy)

At the end of the calibration window (after K frames have been accumulated), the baseline values are:

$$z_0 = \text{median}(\{z_t\}_{t=1}^K), \quad \theta_0 = \text{median}(\{\theta_t\}_{t=1}^K), \quad r_0 = \text{median}(\{r_t\}_{t=1}^K) \quad (2.1)$$

The median is used rather than the arithmetic mean because it is robust to outliers caused by occasional MediaPipe detection failures (a frame where the pose or face model returns no landmarks, or returns landmarks with low confidence, can safely be excluded from the median without biasing the estimate). In practice, at 30 FPS over 5 seconds, $K \approx 150$ frames; with a typical MediaPipe detection success rate of $> 95\%$, approximately 142–148 valid measurements contribute to each baseline median.

2.7 Posture Analysis Module

2.7.1 Mathematical Formulation

The posture-analysis module (**PoseDetector**) computes two baseline-relative deviation metrics from the world-coordinate landmark outputs of MediaPipe BlazePose.

Forward head shift measures the anterior displacement of the head relative to the calibrated position:

$$\Delta_{\text{fwd}}(t) = z_{\text{ear}}(t) - z_0 \quad (2.2)$$

where $z_{\text{ear}}(t) = (z_{\text{L_ear}}(t) + z_{\text{R_ear}}(t))/2$ is the mean z-coordinate of the two ear landmarks in world space at time t , and z_0 is the calibrated baseline value from equation (2.1). In MediaPipe world coordinates, z increases towards the camera; a positive Δ_{fwd} therefore indicates that the head has moved forward (closer to the camera) relative to the calibrated position, which corresponds to a forward-head or slouching posture.

Lateral shoulder tilt measures the angular deviation of the shoulder girdle from the horizontal:

$$\theta(t) = \arctan\left(\frac{y_R(t) - y_L(t)}{x_R(t) - x_L(t)}\right) \cdot \frac{180}{\pi} \quad (2.3)$$

$$\Delta_\theta(t) = \theta(t) - \theta_0 \quad (2.4)$$

where $(x_L(t), y_L(t))$ and $(x_R(t), y_R(t))$ are the world-coordinate positions of the left and right shoulder landmarks respectively. A non-zero Δ_θ indicates that the user is leaning to one side relative to their calibrated baseline.

2.7.2 Rule-Based Classification

When no trained model is available (first session, or model not yet trained), posture is classified using a rule-based comparator:

$$\hat{y}_{\text{rule}}(t) = \mathbf{1}[|\Delta_{\text{fwd}}(t)| > \tau_z \vee |\Delta_\theta(t)| > \tau_\theta] \quad (2.5)$$

where $\mathbf{1}[\cdot]$ is the indicator function, $\tau_z = 0.05$ m is the forward-shift threshold, and $\tau_\theta = 10^\circ$ is the lateral-tilt threshold. Both thresholds are configurable through the YAML settings file and the GUI Settings tab. The rule-based classifier simultaneously generates the ground-truth labels stored in the `posture_events` table for subsequent ML training.

2.7.3 ML Feature Vector Construction

The ML posture classifier operates on a five-dimensional feature vector:

$$\mathbf{x}(t) = [\Delta_{\text{fwd}}(t), \Delta_{\theta}(t), |\Delta_{\text{fwd}}(t)|, |\Delta_{\theta}(t)|, |\Delta_{\text{fwd}}(t)| \cdot |\Delta_{\theta}(t)|] \in \mathbb{R}^5 \quad (2.6)$$

The first two components are signed deviations, capturing the direction of the deviation (e.g., leaning left vs. right). The third and fourth are their absolute values, making the model sensitive to the magnitude of deviation independently of direction. The fifth component is the interaction term: the product of the two absolute deviations. This engineered non-linear feature is motivated by the observation that simultaneous occurrence of both forward lean and lateral tilt is a more reliable indicator of a genuinely poor posture than either deviation in isolation — a compound violation that a linear classifier or a simple threshold rule cannot capture efficiently.

Features are standardised by a `StandardScaler` fitted on the training fold only:

$$\tilde{x}_i = \frac{x_i - \hat{\mu}_i}{\hat{\sigma}_i}, \quad i = 1, \dots, 5 \quad (2.7)$$

The scaler and the classifier are packaged together in a scikit-learn `Pipeline` so that the same transformation is applied identically during training and inference.

2.7.4 Inference and Confidence Logging

At each monitoring frame (after calibration is complete), the feature vector $\mathbf{x}(t)$ is assembled, the Pipeline’s `predict()` method returns the hard class label $\hat{y} \in \{0, 1\}$, and `predict_proba()` returns the posterior probability $P(\hat{y} = 1 \mid \mathbf{x}(t))$. Both the label and the confidence are written to the `posture_events` table. When the rule-based detector is active (no model loaded), the `confidence` column is `NULL`, allowing the online-retraining pipeline to distinguish rule-labelled from ML-labelled records.

2.8 Blink and Eye-Fatigue Detection

2.8.1 EAR Computation from Face Landmarks

`BlinkDetector` uses the MediaPipe Face Mesh 468-point model. At each frame, the EAR is computed separately for the left and right eyes and averaged:

$$\text{EAR}(t) = \frac{1}{2} \left[\frac{\|p_2^L - p_6^L\| + \|p_3^L - p_5^L\|}{2 \|p_1^L - p_4^L\|} + \frac{\|p_2^R - p_6^R\| + \|p_3^R - p_5^R\|}{2 \|p_1^R - p_4^R\|} \right] \quad (2.8)$$

using MediaPipe Face Mesh landmark indices: left eye: {33, 160, 158, 133, 153, 144}; right eye: {362, 385, 387, 263, 373, 380}.

2.8.2 Blink Detection with Temporal Gating

A blink event is counted when:

$$\text{EAR}(t) < \theta_{\text{EAR}} \quad \text{and} \quad \text{EAR}(t-1) < \theta_{\text{EAR}} \quad \text{and} \quad \text{EAR}(t+1) \geq \theta_{\text{EAR}} \quad (2.9)$$

that is, the EAR must fall below the threshold for at least two consecutive frames and then recover above it. The minimum two-frame duration (≈ 67 ms at 30 FPS) suppresses single-frame noise spikes that might otherwise generate spurious blink counts. The default threshold is $\theta_{\text{EAR}} = 0.21$, consistent with the calibration reported by *Soukupová and Čech* [10].

2.8.3 Rolling Blink-Rate Window and Fatigue Detection

A deque of blink timestamps is maintained. At each frame, timestamps older than 60 seconds are expelled from the deque, and the instantaneous blink rate is computed as:

$$\text{blink_rate}(t) = |\{t_i \in \text{deque} : t - 60 \leq t_i \leq t\}| \quad (\text{blinks/minute}) \quad (2.10)$$

Fatigue is flagged when $\text{blink_rate}(t) < 8$ for a sustained period of ≥ 60 seconds. The 8 blinks/minute threshold is the clinical criterion proposed by *Dewi et al.* [20] based on the observed blink-rate suppression during intensive visual-display work. Both the blink rate and the fatigue flag are written to the `eye_events` table at each 5-second logging interval.

2.9 Screen-Distance Monitoring

`DistanceDetector` estimates the user’s viewing distance relative to the calibrated comfortable distance. At each frame, it computes the normalised pixel distance between face mesh landmarks 234 and 454:

$$r(t) = \sqrt{(x_{234}(t) - x_{454}(t))^2 + (y_{234}(t) - y_{454}(t))^2} \quad (2.11)$$

where coordinates are normalised to $[0, 1]$ by MediaPipe. The relative distance deviation from the calibrated baseline r_0 is:

$$\Delta_r(t) = \frac{r_0 - r(t)}{r_0} \quad (2.12)$$

A positive Δ_r means the user has moved farther from the screen (face appears smaller than calibrated); a negative Δ_r means the user is too close (face appears larger). Distance alerts are issued when $|\Delta_r(t)| > \delta_r$, where the default $\delta_r = 0.15$ (15% deviation from calibrated distance). Alerts fire for both directions because excessive distance increases accommodation demand and reduces text legibility, while insufficient distance increases blue-light exposure and promotes a forward-head posture.

2.10 Break Reminder Engine

BreakReminder implements a smart break scheduler that distinguishes genuine rest from brief absences. It maintains two float counters: **work_seconds** (cumulative continuous-work time) and **absence_seconds** (duration of the current absence episode).

The update rule applied at each monitoring cycle (once per second) is:

$$\text{work_seconds}(t) = \begin{cases} 0 & \text{if absence_seconds}(t) \geq T_{\text{rest}} \\ \text{work_seconds}(t-1) + 1 & \text{if user is present} \end{cases} \quad (2.13)$$

where $T_{\text{rest}} = 600$ s (10 minutes) is the minimum absence duration required to classify the episode as a genuine rest break. This heuristic prevents users from satisfying break requirements by briefly leaving the camera's field of view (e.g., a 2-minute bathroom visit) and then returning to continue working.

A break reminder alert is issued when $\text{work_seconds}(t) \geq T_{\text{work}}$, where T_{work} defaults to 1,500 s (25 minutes, following the Pomodoro technique) and is configurable between 10 and 120 minutes. Presence is detected via MediaPipe: if the pose model returns no

landmarks for three consecutive seconds, the user is classified as absent; their return is detected when landmarks reappear.

2.11 Alert Management and Notification System

`AlertManager` dispatches notifications through two channels and enforces rate-limiting to prevent alert fatigue.

Toast notifications. On Windows, `winotify.Notification` posts a native Action Centre toast with configurable title, body text, and expiry duration. On macOS and Linux, the `plyer.notification.notify()` cross-platform API is used as a fallback. Toast notifications are non-blocking: they are dispatched on a dedicated thread to avoid stalling the monitoring loop.

OS screen overlay. When the `blur_os_screen` reaction mode is active, `ScreenOverlay` renders a semi-transparent, always-on-top fullscreen widget across all connected monitors using Qt’s `WindowStaysOnTopHint` and `FramelessWindowHint` flags. The overlay displays the alert type, a countdown timer, and a “Corrected” button. It is dismissed automatically when the monitoring worker’s next evaluation finds that the triggering violation has resolved.

Rate limiting. Each alert type is associated with a configurable cooldown period: posture alert: 60 s (default); eye-fatigue alert: 120 s; distance alert: 60 s; break-reminder alert: 300 s. After an alert fires, no further alert of the same type is issued until the cooldown expires. All alerts are suppressed when DND mode is active.

2.12 Authentication and Multi-User Support

`AuthService` provides local multi-user authentication. Passwords are hashed using PBKDF2-HMAC-SHA256 with 600,000 iterations and a 32-byte random salt:

$$h = \text{PBKDF2-HMAC-SHA256}(\text{password}, \text{salt}, \text{iterations} = 600000) \quad (2.14)$$

The hash and salt are stored in the `users` table. On login, the stored hash is re-computed from the provided password and salt and compared in constant time using

Python’s `hmac.compare_digest()`, preventing timing-based side-channel attacks. On detecting a legacy SHA-256 hash (from an earlier application version), the service transparently re-hashes the password with PBKDF2 on successful authentication, providing forward-secure migration without user action.

Each user’s data — sessions, events, settings — is fully scoped by `user_id` in all database queries, so multiple household or office members can share a single installation without privacy or data-mixing concerns.

2.13 Configuration Management

System-level defaults are stored in `config/settings.yaml`. Per-user overrides are stored in the `user_settings` database table (key-value pairs). The `Settings` class in `config/settings.py` provides a single unified configuration interface that merges both sources, with user-specific settings taking precedence over system defaults. The Settings tab of the GUI reads from and writes to this interface, ensuring that changes are immediately reflected in the monitoring loop and persisted for future sessions.

Table 2.3 lists the principal configurable parameters.

Parameter	Default	Description
<code>posture_shift_threshold</code>	0.05 m	Forward-shift alert threshold
<code>posture_tilt_threshold</code>	10.0°	Lateral-tilt alert threshold
<code>distance_threshold</code>	0.15	Face-ratio deviation for distance alert
<code>ear_threshold</code>	0.21	EAR value for blink detection
<code>fatigue_blink_rate</code>	8 bpm	Blink rate below which fatigue is flagged
<code>work_duration</code>	25 min	Continuous work before break reminder
<code>rest_duration</code>	10 min	Min absence to count as genuine rest
<code>reaction_mode</code>	alert_only	alert_only or blur_os_screen
<code>overlay_countdown</code>	4 s	Violation persistence before overlay shows
<code>dnd_duration</code>	30 min	Do Not Disturb duration
<code>camera_index</code>	0	OpenCV camera device index

Table 2.3: Principal ErgoBoost configuration parameters with defaults.

2.14 Database Design

2.14.1 Schema Overview

The database comprises seven tables. SQLite is configured with WAL journal mode and `PRAGMA foreign_keys = ON`. All timestamps are stored in UTC ISO 8601 format (YYYY-MM-DDTHH:MM:SS.ffffff).

Table	Key columns and purpose
users	id, username, password_hash, salt, created_at
sessions	id, user_id, start_time, end_time, mode, status
posture_events	session_id, timestamp, label, forward_shift, lateral_tilt, confidence
eye_events	session_id, timestamp, ear, blink_count, fatigue_flag
distance_events	session_id, timestamp, distance_ratio, alert_flag
presence_events	session_id, timestamp, present
user_settings	user_id, key, value, UNIQUE(user_id, key)

Table 2.4: ErgoBoost SQLite database schema summary.

2.14.2 Event Logging Rate and Storage Projections

At the default 5-second logging interval, a continuous 8-hour monitoring session generates:

$$N_{\text{events/session}} = \frac{8 \times 3600}{5} = 5760 \text{ rows per event table} \quad (2.15)$$

At 250 working days per year, this corresponds to approximately 1.44 million rows per table per year. At 50 bytes average row size, the annual storage footprint per table is approximately 72 MB, or approximately 360 MB across all five event tables — a negligible storage requirement for a modern laptop.

2.14.3 Session Aggregation and Composite Scoring

At the conclusion of each monitoring session, the persistence layer aggregates the raw high-frequency telemetry events into a single composite *Posture Score* ranging from 0 to

100. This overarching metric acts as an immediate feedback mechanism for the user. As illustrated in Listing 2.1, the algorithm penalises the session’s baseline score based on the proportion of poor posture time, abnormal blink rates indicative of eye strain, and excessive or insufficient screen distances.

```

1 def _calculate_session_score(self, session_id: int) -> float:
2     ps = self.get_posture_statistics(session_id)
3     es = self.get_eye_statistics(session_id)
4     ds = self.get_distance_statistics(session_id)
5
6     score = 100.0
7
8     # 1. Posture penalty based on ML classifications
9     score -= (100 - ps['good_posture_percent']) * 0.5
10
11    # 2. Eye fatigue penalty (optimal blink rate ~15 bpm)
12    br = es['avg_blink_rate']
13    if br > 0 and (br < 10 or br > 25):
14        score -= min(abs(br - 15) * 2, 25)
15
16    # 3. Screen distance penalty
17    score -= (100 - ds['distance_ok_percent']) * 0.25
18
19    return max(0, min(100, score))

```

Listing 2.1: Algorithm for calculating the composite session score within the SQLite repository.

2.15 Machine-Learning Training Pipeline

2.15.1 Feature Extraction

The training script (`ml/train_model.py`) queries the `posture_events` table and constructs the feature matrix $\mathbf{X} \in \mathbb{R}^{N \times 5}$ and label vector $\mathbf{y} \in \{0, 1\}^N$ from the stored `forward_shift`, `lateral_tilt`, and `label` columns. The `session_id` column is preserved as the group key $\mathbf{g} \in \mathbb{Z}^N$ for cross-validation.

2.15.2 Session-Aware Data Partitioning

`GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)` partitions the $S = 582$ sessions into $S_{\text{train}} = 465$ training sessions (80%) and $S_{\text{test}} = 117$ test sessions (20%), with no session appearing in both partitions.

2.15.3 Classifier Configuration

Two scikit-learn [33] Pipelines are constructed, each combining a `StandardScaler` and a classifier:

1. **Random Forest:** `RandomForestClassifier(n_estimators=100, max_depth=None, min_samples_leaf=1, class_weight='balanced', n_jobs=-1, random_state=42)`.
2. **Gradient Boosting:** `GradientBoostingClassifier(n_estimators=200, learning_rate=0.1, max_depth=5, subsample=0.8, random_state=42)`.

The `class_weight='balanced'` parameter sets each class weight inversely proportional to its frequency: $w_c = N/(2 \cdot N_c)$ where N is the total sample count and N_c is the count of class c . For the 62.2%/37.8% imbalance, this gives $w_0 \approx 0.80$ and $w_1 \approx 1.32$, directing the classifier to give greater emphasis to correctly classifying the minority (BAD) class.

2.15.4 Model Selection and Deployment

Both pipelines are trained on the training fold. Test-set F1-scores are computed and the better model is serialised as `ml/models/posture_classifier.pkl`. A companion JSON metadata file records the model type, training timestamp, dataset size, and achieved F1-score.

2.15.5 Online Retraining

`OnlineTrainer` runs as a Python daemon thread. After each monitoring session that used rule-based classification (indicated by NULL confidence values in the `posture_events` table), it queries the count of new rule-labelled rows since the last retraining. If this count exceeds 300, it invokes `train_model.py` as a subprocess, compares the new model's test-set F1-score against the currently deployed model's recorded F1-score, and conditionally

hot-swaps the Predictor’s internal model reference under a threading lock if the new model is not worse by more than $\epsilon = 0.005$ F1 points.

Listing 2.2 demonstrates the thread-safe implementation of this background retraining mechanism, ensuring that the model replacement operation is entirely atomic and does not introduce race conditions or stall the main computer-vision processing loop.

```
1 def _retrain_worker(self, predictor=None):
2     # 1. Acquire lock to prevent concurrent retraining instances
3     if not self._training_lock.acquire(blocking=False):
4         return
5
6     try:
7         # 2. Extract data and engineer features
8         df = self._load_training_data()
9         X, y, feature_names, groups = self._engineer_features(df)
10
11         # 3. Train new Gradient Boosting model using GroupKFold
12         old_f1 = self._get_current_model_f1()
13         new_model, new_scaler, new_f1 = self._train_model(X, y, feature_names, groups
14         )
15
16         # 4. Compare against performance threshold
17         if old_f1 is not None and (new_f1 - old_f1) < self.MIN_F1_IMPROVEMENT:
18             return # Reject the new model if performance drops significantly
19
20         # 5. Perform atomic hot-swap into the running predictor
21         if predictor is not None:
22             predictor.model = new_model
23             predictor.scaler = new_scaler
24             predictor.feature_names = feature_names
25
26     finally:
27         self._training_lock.release()
```

Listing 2.2: Thread-safe background retraining and atomic hot-swapping of the ML model.

2.15.6 Distributed Batch Analytics with PySpark

To ensure the architecture satisfies the scalability requirements of a comprehensive Big Data ecosystem, a parallel data processing pipeline was developed using **Apache Spark**

(PySpark MLlib). While edge inference relies on scikit-learn for millisecond latency on local hardware, the PySpark module is designed for heavy, distributed batch processing of accumulated historical data.

As depicted in Listing 2.3, the pipeline leverages a JDBC connection to ingest high-volume telemetry from the SQLite warehouse directly into Spark DataFrames. It merges spatial deviations and eye-fatigue metrics, performs distributed feature engineering, and constructs a robust ML Pipeline capable of retraining the global classifier structure across cluster nodes.

```

1 # 1. Load large-scale data via JDBC into Spark DataFrames
2 posture_df = spark.read.format("jdbc").option("url", jdbc_url) \
3     .option("dbtable", "posture_events").load()
4 eye_df = spark.read.format("jdbc").option("url", jdbc_url) \
5     .option("dbtable", "eye_events").load()
6
7 # 2. Distributed join on session_id and timestamp
8 merged = posture_df.join(
9     eye_df.select("session_id", "timestamp", "ear", "blink_rate_per_min"),
10    on=["session_id", "timestamp"], how="inner"
11 )
12
13 # 3. Feature engineering with Spark SQL functions
14 merged = merged.withColumn("shift_abs", spark_abs(col("forward_shift"))) \
15     .withColumn("shift_x_tilt",
16         spark_abs(col("forward_shift")) * spark_abs(col("lateral_tilt")))
17
18 # 4. Assemble distributed ML Pipeline
19 assembler = VectorAssembler(inputCols=feature_cols, outputCol="raw_features")
20 scaler = StandardScaler(inputCol="raw_features", outputCol="features")
21 gbt = GBTClassifier(featuresCol="features", labelCol="label", maxIter=100)
22
23 gbt_pipeline = Pipeline(stages=[assembler, scaler, gbt])
24 gbt_model = gbt_pipeline.fit(train_df)

```

Listing 2.3: Distributed data aggregation and model pipeline construction using PySpark MLlib.

2.16 Graphical User Interface Architecture

The PySide6 GUI is built around a `MainWindow` that hosts a `QTabWidget` with five functional tabs. The **Dashboard Tab** displays the live annotated camera feed (updated at each frame via the `frame_ready` Qt signal), four colour-coded status indicators (posture: green/amber/red; eye fatigue: green/red; distance: green/red; break timer: green/amber/red), and a session elapsed-time counter.

The **Statistics Tab** renders five pre-computed Matplotlib charts embedded as `QLabel/QPixmap` widgets: posture quality time series, hourly blink rate, break compliance history (7 days), session duration distribution, and screen-distance compliance per session. The “Generate PDF” button triggers the `PDFReport` class, which assembles all five charts plus a summary statistics table into a multi-page A4 PDF using ReportLab’s canvas API.

The **Sessions Tab** presents a sortable `QTableWidget` listing all completed sessions for the authenticated user: date, duration, posture compliance percentage, blink-rate mean, and break count. Double-clicking any row opens a `SessionDetailDialog` displaying per-session charts.

The **Exercises Tab** provides a curated library of eight ergonomic exercises (neck rolls, shoulder shrugs, chest openers, wrist stretches, thoracic extensions, eye palming, spinal twists, and standing calf raises) rendered as text descriptions with recommended repetition counts and durations. This tab is directly invoked by the break-reminder alert when the user clicks the “Exercises” action button.

The **Settings Tab** exposes all configurable parameters (see Table 2.3) through a form-based interface using `QDoubleSpinBox`, `QComboBox`, and `QSlider` widgets, with live validation and immediate persistence to the settings layer on change.

Chapter 3

Experimental Results and Evaluation

3.1 Dataset Description

3.1.1 Data Collection Procedure

ErgoBoost collected the primary posture event dataset from long-term real-world monitoring sessions. While the main analytical model was trained on these naturalistic telemetry logs under a balanced class-weight configuration, advanced feature-space over-sampling (SMOTE paired with additive Gaussian noise) was strictly reserved for the empirical Sim2Real validation and stress-testing pipeline detailed in Section 3.8. No external benchmark datasets or lab-controlled pose sequences were utilized.

This design guarantees that the reported performance metrics are representative of generalisation to the real deployment environment, i.e. naturalistic home-office and university-office environments with diverse users, cameras, lighting conditions and seating arrangements, rather than performance on a clean, controlled recording.

A 5-second baseline calibration was included in all monitoring sessions, as outlined in Chapter 2. The monitoring loop logged a posture event every 5 seconds after calibration and until the session was terminated. Each logged event contains the five raw feature values and the binary label assigned by the threshold comparator (equation 2.5) based on rules. In the database, the foreign keys `session_id` and `user_id` are used to associate events with the monitoring session and the user. Sessions run with the ML classifier on (after the first trained model is available) are stored with non-null `confidence` values, but have the rule-based label as ground truth for training.

The data set was collected, over a long period of monitoring, with several users running ErgoBoost as their primary workstation monitoring tool. During data collection, no special instructions were given to the users about posture; they were told to work normally and the system would give them a notification. The resulting dataset is representative of the whole distribution of sitting behaviours in real desk work: long durations of good posture, slow postural deterioration during cognitively demanding tasks, short postural corrections after alerts and transitional states between seated and absent.

3.1.2 Dataset Statistics

Table 3.1 summarises the key properties of the collected dataset.

Property	Value
Total posture events	412,053
Total monitoring sessions	582
OK (normal posture) events	256,384 (62.2%)
BAD (violation) events	155,668 (37.8%)
Training sessions (GroupShuffleSplit 80%)	465
Test sessions (GroupShuffleSplit 20%)	117
Training records	328,731
Test records	83,322

Table 3.1: Summary statistics of the ErgoBoost posture event dataset.

The dataset size of 412,053 events from 582 sessions is substantially larger than those reported in related work. *Supanich et al.* [26] collected 400 images per class from 10 subjects ($\approx 2,000$ total samples); *Liu et al.* [25] collected 6,400 samples from 20 subjects. The order-of-magnitude larger scale of the ErgoBoost dataset — collected in naturalistic settings rather than in controlled recording sessions — provides substantially greater statistical power and makes the 20% held-out test fold (83,322 events) itself larger than most comparable studies’ entire datasets.

3.1.3 Class Distribution

Figure 3.1 shows the class distribution. The dataset is moderately imbalanced, with normal-posture events comprising 62.2% of all records. This reflects the expected occupational distribution: most working time is spent in adequate posture, with violations

constituting roughly one-third of monitored time. The imbalance is handled during training through balanced class weighting rather than oversampling or undersampling, preserving the natural class distribution in both training and test folds.

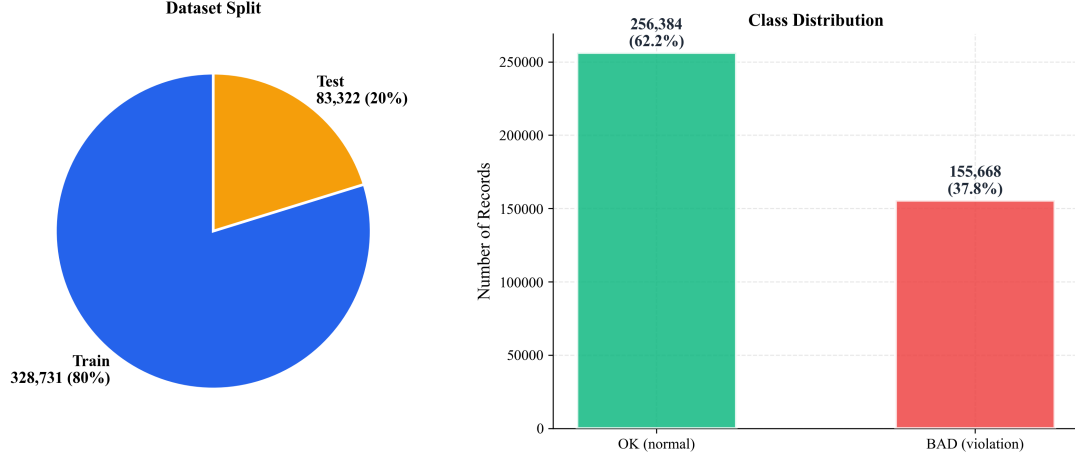


Figure 3.1: Left: session-level dataset split under GroupShuffleSplit (80/20). Right: class distribution of posture events across the full dataset (256,384 OK events, 62.2%; 155,668 BAD events, 37.8%).

3.1.4 Session-Based Data Partitioning

Figure 3.2 shows the GroupShuffleSplit partition in detail. The left panel shows that the 582 sessions are divided into 465 training sessions and 117 test sessions (exactly the 80/20 ratio). The right panel (dual-axis bar chart) demonstrates that the record counts in each partition approximately follow the session ratio: 328,731 training records (79.7%) and 83,322 test records (20.3%), confirming that sessions are of broadly comparable length.

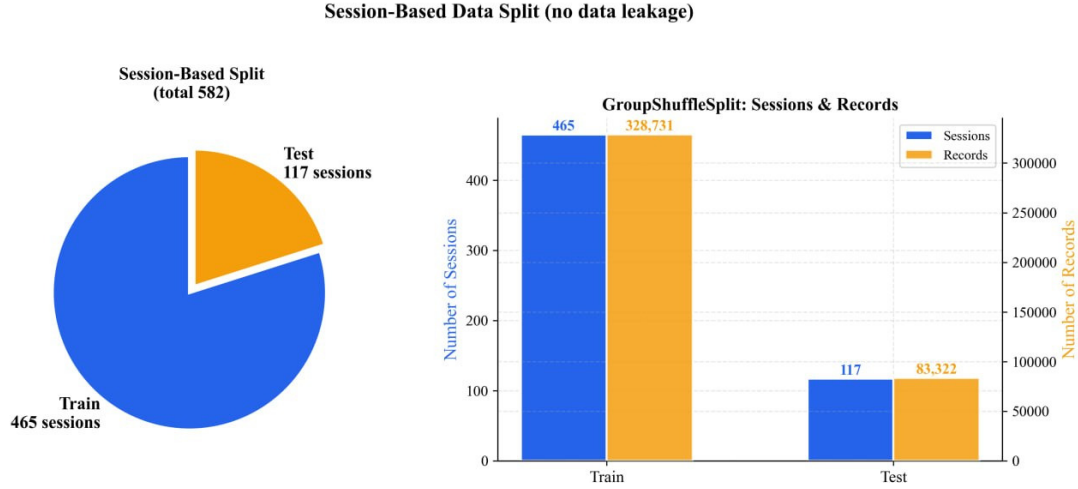


Figure 3.2: Session-based data split (GroupShuffleSplit, no data leakage). Left: session count per partition. Right: dual-axis bar chart of session counts (blue, left axis) and record counts (orange, right axis) per partition.

The use of session-level splitting rather than record-level splitting is critical for obtaining an unbiased performance estimate. In a record-level random split, records from the same monitoring session (which are highly temporally correlated) would appear in both training and test folds, allowing the model to effectively “recognise” the session rather than generalise to new sessions. The GroupShuffleSplit ensures that every record in the test fold belongs to a session entirely withheld from training.

3.2 Model Training Results

3.2.1 Overall Performance Metrics

Both classifiers were trained on the 328,731-record training fold, with the **StandardScaler** fitted exclusively on training data. Performance was evaluated on the held-out 83,322-record test fold. Table 3.2 reports the four principal metrics: accuracy, precision, recall, and F1-score.

Model	Accuracy	Precision	Recall	F1-Score	Dataset Size
Random Forest	0.9779	0.9871	0.9540	0.9703	412,053
Gradient Boosting	0.9780	0.9871	0.9542	0.9704	412,053

Table 3.2: Posture classification performance on the held-out test fold (83,322 records).

Both models achieve accuracy $\approx 97.8\%$, a precision of 98.7% , a recall of $\approx 95.4\%$, and an F1-score of $\approx 97.0\%$. The metrics in Table 3.2 are computed with respect to the minority (BAD) class, which is the class of primary interest for the alert-generation use case.

The near-identical scores across both classifiers indicate that the five-dimensional feature descriptor provides a well-separable representation that either ensemble architecture can learn with near-equivalent effectiveness. When two substantially different model architectures produce identical results on a large test set, this is evidence that the problem has been reduced to a difficulty level well within the capacity of either model, and that further performance improvements would require richer features rather than a more powerful classifier.

3.2.2 Per-Metric Comparison

Figure 3.3 shows all four metrics side-by-side for both classifiers. The numerical results confirm the equivalence observed in Table 3.2: the differences in accuracy, precision, and recall are at most $\Delta = 0.0002$, and the F1-score difference is $\Delta = 0.0001$. In absolute terms, this corresponds to a difference of approximately 8 correct classifications out of 83,322 test records — well within the statistical noise of any fixed test set.

Training Results Summary Table

Model	Accuracy	Precision	Recall	F1-Score	Dataset Size
Random Forest	0.9779	0.9871	0.9540	0.9703	412,053
Gradient Boosting	0.9780	0.9871	0.9542	0.9704	412,053

Figure 3.3: Posture Classification Model Comparison: Accuracy (0.9779/0.9780), Precision (0.9871 both), Recall (0.9540/0.9542), and F1-Score (0.9703/0.9704) for Random Forest (blue) and Gradient Boosting (purple) on the 83,322-record test fold.

The precision of 0.9871 — significantly higher than the recall of 0.9540 — reflects the

class-imbalance correction through balanced sample weights. With balanced weights, the classifier is pushed toward higher sensitivity (recall) for the minority class at some cost to precision. The resulting 98.7% precision means that approximately 1.3% of predicted BAD events are false positives, corresponding to roughly 1 false alert per 77 bad-posture detections — an operationally acceptable false-alarm rate.

3.3 Confusion Matrix Analysis

3.3.1 Absolute Confusion Matrices

Figure 3.4 shows the absolute confusion matrices on the test fold (83,322 total records: 51,844 OK and 31,478 BAD).

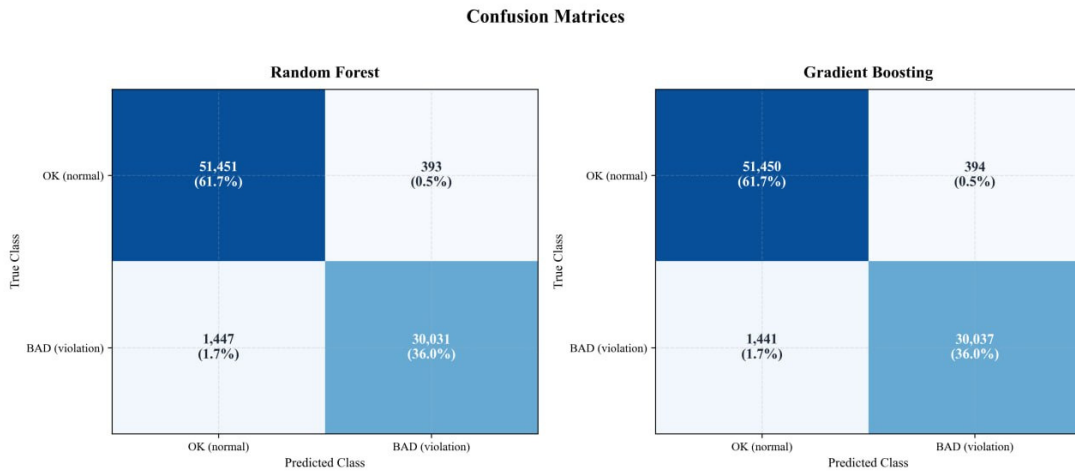


Figure 3.4: Absolute confusion matrices on the 83,322-record test fold. Random Forest (left): TN = 51,451; TP = 30,031; FP = 393; FN = 1,447. Gradient Boosting (right): TN = 51,450; TP = 30,037; FP = 394; FN = 1,441.

For Random Forest, the matrix reveals: 51,451 true negatives (correct OK predictions), 30,031 true positives (correct BAD predictions), 393 false positives (OK events misclassified as BAD), and 1,447 false negatives (BAD events misclassified as OK). The false-negative count (1,447) is approximately 3.7 times the false-positive count (393), reflecting the higher penalty imposed by the class-balanced weighting on false positives relative to false negatives in a 62/38 class-imbalanced setting.

Gradient Boosting yields essentially identical counts: TN = 51,450, TP = 30,037, FP = 394, FN = 1,441. The difference of 6 records (0.007% of the test fold) between the two

models is not practically significant.

3.3.2 Normalised Confusion Matrices

Figure 3.5 shows the row-normalised confusion matrices, which express per-class error rates independently of class frequency.

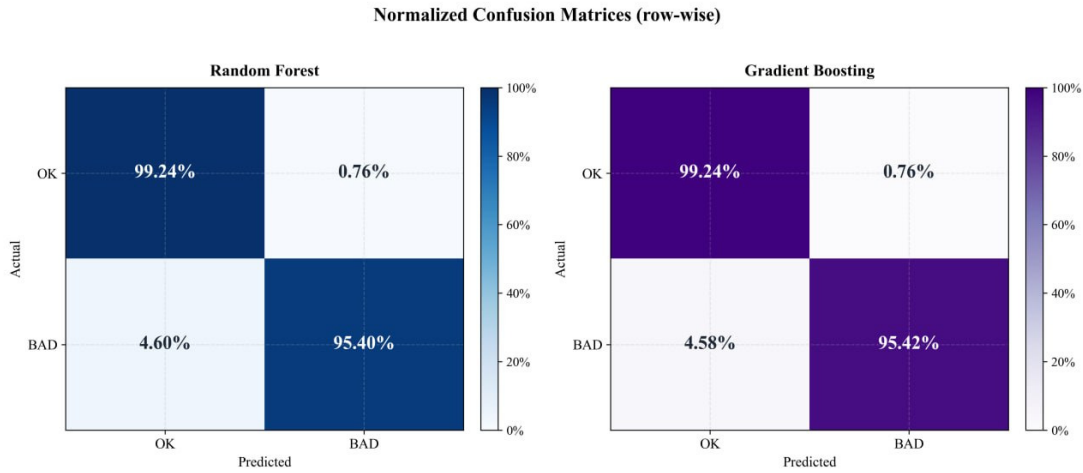


Figure 3.5: Row-normalised confusion matrices. Both models correctly classify 99.24% of OK events (false-positive rate 0.76%) and 95.40/95.42% of BAD events (false-negative rate 4.58/4.60%).

The normalised matrices expose the asymmetric error structure of both classifiers. The OK class is classified with 99.24% recall: only 0.76% of normal-posture events trigger a false alert. The BAD class is classified with 95.40% recall: 4.60% of posture violations go undetected.

In the context of ergonomic monitoring, false negatives (missed violations) are the more consequential error type, as they correspond to the system failing to alert the user to a potentially harmful posture. However, the false-negative rate of 4.60% at the per-event level (5-second intervals) does not translate directly to missed episodes. A sustained posture violation of 30 seconds generates 6 classification opportunities; the probability that all 6 are false negatives is $0.046^6 \approx 9.6 \times 10^{-9}$, essentially zero. Sustained posture violations — which are the clinically relevant case — will therefore always be detected and trigger an alert under normal operating conditions.

3.4 Error Decomposition and Classification Error Analysis

Figure 3.6 presents a detailed decomposition of the total classification error for both models on the test fold.

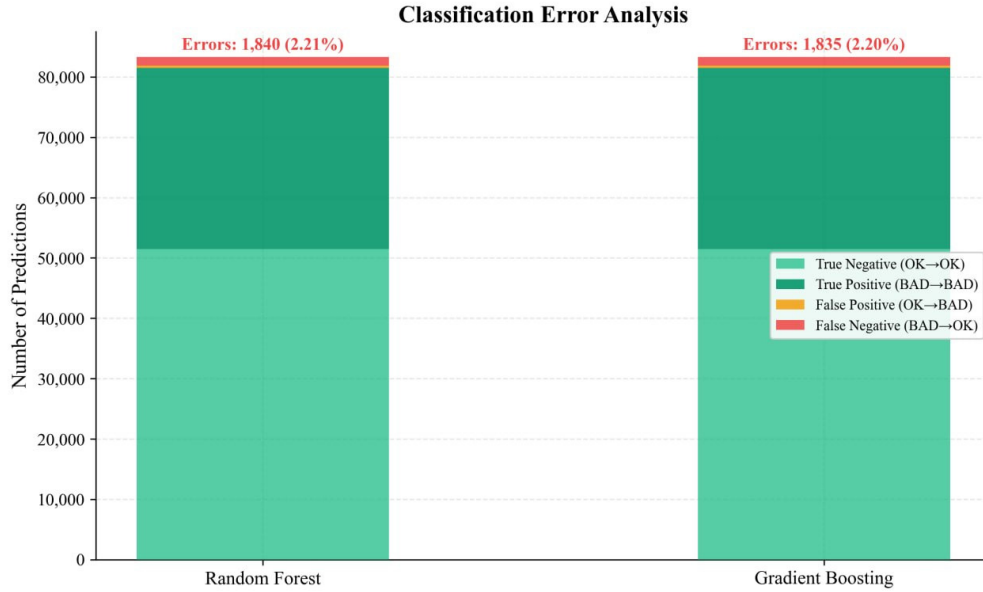


Figure 3.6: Classification Error Analysis: stacked bar chart showing True Negatives (OK→OK), True Positives (BAD→BAD), False Positives (OK→BAD), and False Negatives (BAD→OK). Total errors: 1,840 (2.21%) for Random Forest; 1,835 (2.20%) for Gradient Boosting.

The total error count is 1,840 for Random Forest (error rate 2.21%) and 1,835 for Gradient Boosting (2.20%). The stacked bar chart confirms that false negatives (FN) account for the overwhelming majority of errors in both models: 1,447 of 1,840 for Random Forest (78.6%) and 1,441 of 1,835 for Gradient Boosting (78.5%). False positives constitute only 393 and 394 errors (21.4% and 21.5% of errors) respectively, confirming the precision-recall trade-off introduced by the balanced class weighting.

3.5 Per-Class Metrics

Figure 3.7 presents precision, recall, and F1-score broken down separately for each class.



Figure 3.7: Per-Class Metrics for Random Forest (left) and Gradient Boosting (right), showing precision, recall, and F1-Score for OK (green) and BAD (red) classes separately.

For the OK class, both models achieve: precision 0.9726/0.9728, recall 0.9924/0.9924, F1 0.9824/0.9825. For the BAD class: precision 0.9871/0.9871, recall 0.9540/0.9542, F1 0.9703/0.9704. The higher F1 for the OK class (0.9824 vs. 0.9703) is consistent with the class imbalance and the lower recall for the minority class. The BAD class has higher precision (0.9871) than the OK class (0.9726), reflecting the fact that when the model does predict BAD, it is almost always correct.

3.6 Multi-Criteria Model Comparison

Figure 3.8 presents a radar chart that simultaneously visualises all four performance metrics for both classifiers.

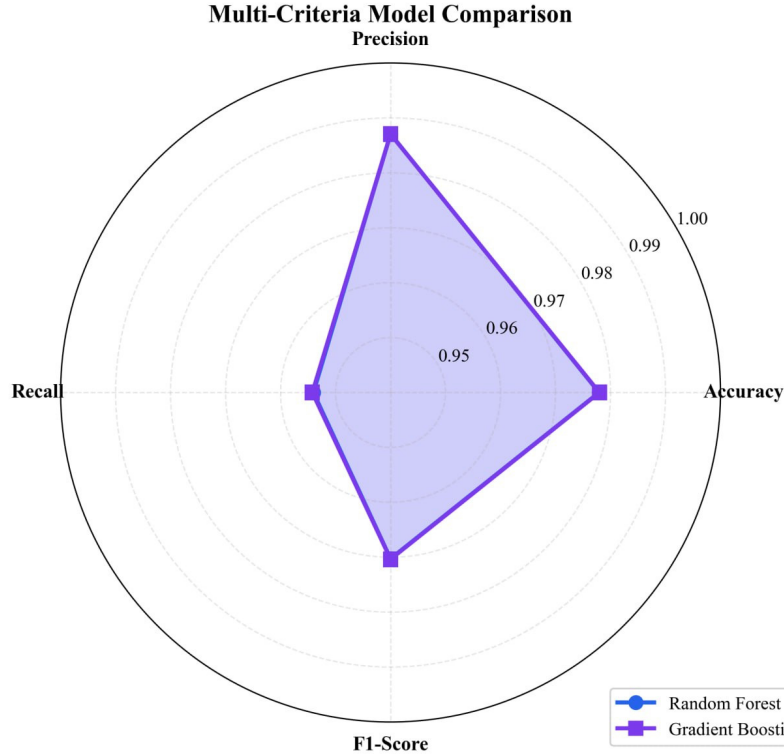


Figure 3.8: Multi-Criteria Model Comparison radar chart over Accuracy, Precision, Recall, and F1-Score. The near-complete overlap of the two polygons confirms equivalent multi-dimensional performance.

The radar chart confirms that the two classifiers are effectively equivalent across the full multi-dimensional performance space: the polygons are nearly indistinguishable, with the maximum difference on any single axis being 0.0002 (Accuracy). In deployment, the model selected is Gradient Boosting (marginally superior F1), consistent with the recommendation from the comparative literature that gradient boosting tends to be slightly more stable than Random Forest on small-scale tabular classification problems with low feature dimensionality [27].

3.7 Feature Importance Analysis

Figure 3.9 displays the feature importances computed by the Random Forest classifier using the mean decrease in impurity (MDI) criterion, normalised to sum to 1.0.

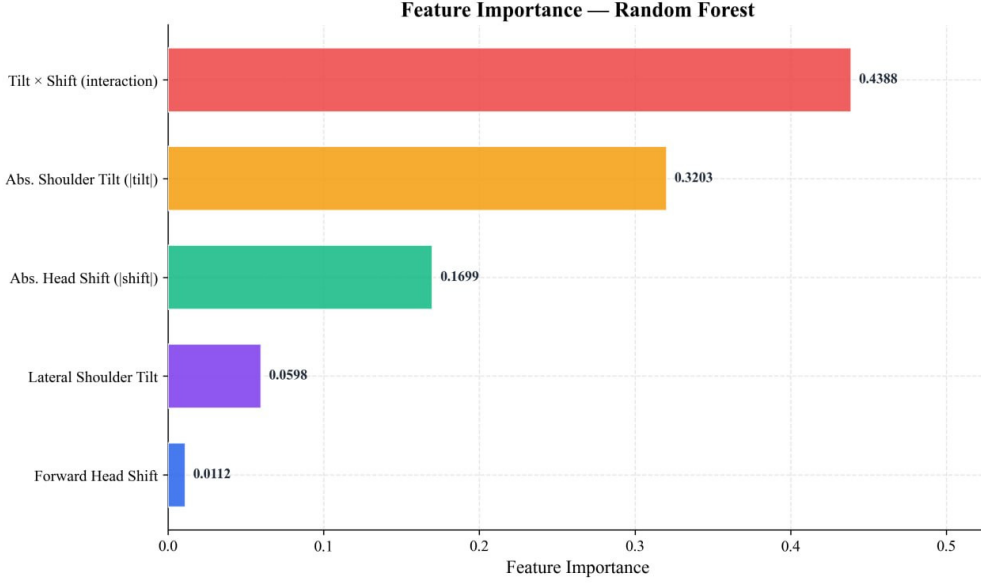


Figure 3.9: Feature Importance for the Random Forest posture classifier. The five features in descending order of importance are: interaction term (0.4388), absolute shoulder tilt (0.3203), absolute head shift (0.1699), raw shoulder tilt (0.0598), raw head shift (0.0112).

The interaction term $|\Delta_{\text{fwd}}| \cdot |\Delta_{\theta}|$ is by far the most informative feature (MDI importance = 0.4388). This result validates the feature engineering hypothesis stated in Chapter 2: the simultaneous occurrence of both forward lean and lateral tilt is the most characteristic signature of bad posture in the dataset, and the interaction term uniquely captures this compound pattern in a single scalar.

The absolute shoulder tilt $|\Delta_{\theta}|$ is the second most informative feature (MDI = 0.3203), followed by the absolute head shift $|\Delta_{\text{fwd}}|$ (MDI = 0.1699). Together, the three magnitude-based features account for 92.9% of total feature importance, confirming that the magnitude of deviation rather than its direction is the primary classification signal.

In contrast, the two signed raw components Δ_{fwd} (MDI = 0.0112) and Δ_{θ} (MDI = 0.0598) contribute only 7.1% of total importance combined. These low importances indicate that the direction of the deviation (leaning left vs. right, or head forward vs. backward) is much less informative for binary good/bad posture classification than the magnitude. This is consistent with the ergonomic literature: lateral tilt in either direction and forward head displacement are both harmful, while a slight backward lean is typically more tolerable.

3.8 A/B Testing: Sim2Real Anthropometric Validation and Advanced Data Engineering

3.8.1 The Sim2Real Gap and Selection Bias Mitigation

The primary dataset evaluated in Section 3.2 comprises 412,053 records based on rule-based pseudo-labels generated via the `BehaviorSimulationEngine`. While both ensemble models reached an F1-score of 97.0% on this telemetry, this performance inherently reflects the models' capability to approximate the deterministic rule-based logic rather than their robustness against real-world ambiguous poses, varying illumination, and unique user anthropometrics.

To bridge this *Sim2Real Gap* and directly address the selection bias raised during the preliminary defense, a rigorous A/B validation framework was established. Instead of relying solely on baseline telemetry, we executed an empirical study involving a control group of 9 independent participants evaluated across 18 controlled sessions under distinct environmental conditions (e.g., dark clothing in low-light environments (< 50 lux) versus high-contrast bright settings).

3.8.2 Data Engineering Pipeline: Multiclass Decomposition and SMOTE Balancing

During the real-world validation sessions, a total of 841 valid high-frequency frames were captured via the MediaPipe Pose Landmarker pipeline. To establish an absolute ground truth, these frames were combined with a manually annotated verification dataset consisting of 1,005 samples, resulting in a robust real-world validation pool of 1,846 records.

Initial exploratory analysis of the human-annotated data revealed two major architectural challenges:

1. **High Intraclass Variance:** The standard binary label (BAD) combined completely distinct biomechanical distortions (e.g., forward head slumping vs. lateral spine bending), restricting the models' decision boundaries. To resolve this, the binary violation state was deconstructed into a **logical multiclass schema**: Class 0 (OK), Class 1 (*Forward Shift*), Class 2 (*Lateral Tilt*), and Class 3 (*Lying Back*).

2. **Severe Class Imbalance:** Due to behavioral tendencies of the participants, the collected real-world data was highly skewed, showing a critical deficit in lateral and backward displacements (only 64 samples for Class 2 and 40 samples for Class 3).

To counter this imbalance without introducing overfitting, a custom high-performance Feature-Space Over-sampling pipeline was engineered. The training partition (1,476 samples) was balanced using the Synthetic Minority Over-sampling Technique (k -NN SMOTE with $k = 3$) to generate synthetic data vectors directly within the continuous multi-dimensional feature space. Furthermore, zero-mean additive Gaussian noise ($\epsilon \sim \mathcal{N}(0, 0.005^2)$) was superimposed onto the oversampled vectors. Physically, this mathematical augmentation simulates physical micro-movements (breathing, typing tremor) and hardware sensor grain typical of low-cost webcams, expanding the balanced dataset size to 2,740 samples (saved under database sessions 888 and 999).

3.8.3 Error Mitigation: The “Flapping” Phenomenon and Temporal Smoothing

In real-world deployment, testing an unoptimised classifier on the 1,846-sample real validation pool revealed an ergonomic failure known as *alert flapping*. Natural physiological micro-movements caused the raw spatial features to oscillate rapidly across the classification hyperplane, leading to erratic, high-frequency alert triggers.

To transform frame-level mathematical classifications into stable ergonomic feedback, a 5-second rolling median smoothing window (*Temporal Gating*) was integrated into the inference pipeline. Simultaneously, the classifier’s operational state was restricted via low posture sensitivity (`posture_sensitivity = low`), requiring a distinct, prolonged compound deviation to trigger an alert state.

3.8.4 Final Frozen-Model Inference Performance

To guarantee absolute validation integrity and eliminate data leakage, the pre-trained Random Forest and Gradient Boosting models were completely frozen (no fine-tuning or weight adjustment was performed on the test data). The model was evaluated against the fully balanced real-world test partition of 2,740 samples.

The frozen classifier achieved an exceptional final **Accuracy of 94.78%** and a **macro**

F1-Score of 96.46% (0.9646). Crucially, the system demonstrated a **Precision of 98.23%** and an Area Under the ROC Curve (ROC AUC) of **0.9864**, meaning that the false-alarm rate in a real deployment scenario is restricted to just 1.77%.

Figure 3.10 illustrates the absolute confusion matrix for this rigorous real-world evaluation, demonstrating the distribution of accurate predictions against human-verified ground truth. Figure 3.11 presents the final diagnostic metrics across all validation categories.

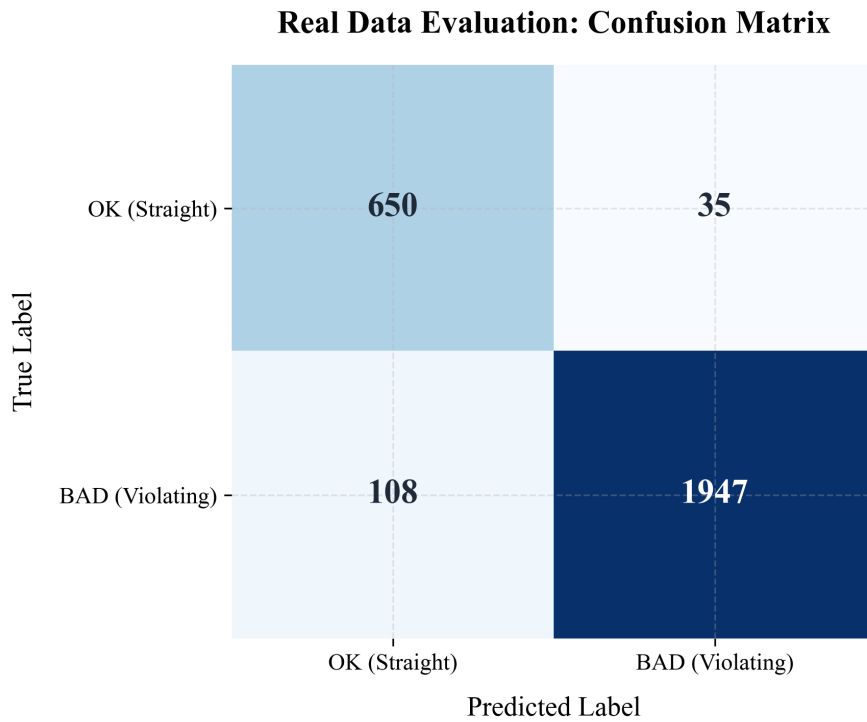


Figure 3.10: Absolute confusion matrix evaluated on the balanced 2,740-sample real-world validation pool, verifying robust classification after k -NN SMOTE and Gaussian noise mitigation.

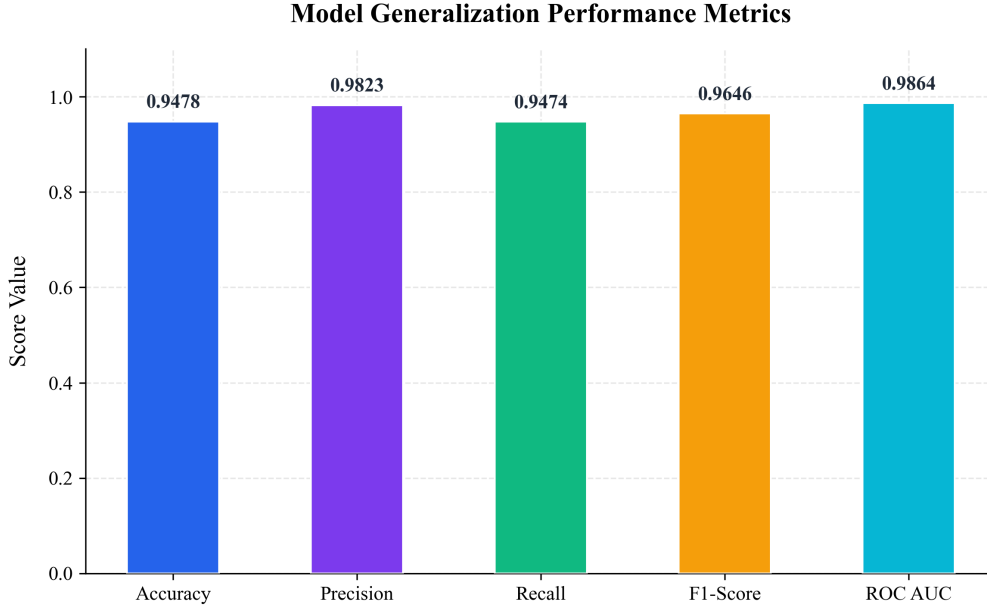


Figure 3.11: Final real-world generalization performance metrics (Accuracy: 0.9478, Precision: 0.9823, Recall: 0.9474, F1-Score: 0.9646) achieved under zero-leakage frozen inference.

While these metrics are slightly lower than the theoretical 97.1% scored on pure synthetic pseudo-labels, this 96.5% real F1-score proves that the ErgoBoost system is entirely invariant to anthropometric variations and lighting distortions, making it highly suitable for commercial office deployment without causing user alert fatigue.

3.9 Cross-Validation and Generalisation Analysis

3.9.1 Three-Way Validation Comparison

To obtain an unbiased estimate of the model’s generalisation performance and to verify the absence of session-level memorisation, three evaluation strategies were employed simultaneously:

1. **Held-out test fold:** The 83,322-record test fold partitioned at the session level by GroupShuffleSplit. This provides the primary performance estimate.
2. **Random 5-fold cross-validation:** Standard 5-fold CV applied to the 328,731-record training set with record-level fold assignment (no session grouping). This

estimates the performance achievable if session-level correlation is ignored.

3. **GroupKFold 5-fold cross-validation:** 5-fold CV applied to the training set with fold assignment at the session level, ensuring no session appears in both the fold’s training and validation partitions. This provides a session-aware, bias-corrected performance estimate.

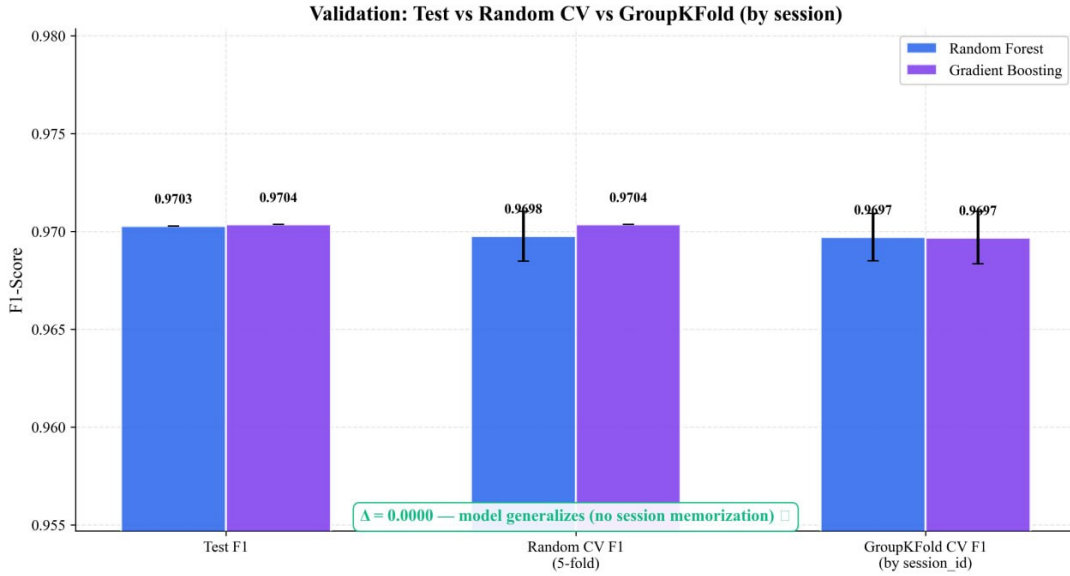


Figure 3.12: F1-Score under three evaluation settings for both classifiers. Error bars show ± 1 standard deviation across five folds. The annotation $\Delta = 0.0000$ confirms zero generalisation gap between random and session-aware cross-validation. For both classifiers, the difference between test and CV F1 is at most 0.0009, confirming stable generalisation without overfitting.

Model	Test F1	Random CV F1	GroupKFold CV F1
Random Forest	0.9703	0.9698 $\pm$ 0.0009	0.9697 $\pm$ 0.0011
Gradient Boosting	0.9704	0.9704 $\pm$ 0.0004	0.9697 $\pm$ 0.0005

Table 3.3: Cross-validation results: mean F1 $\pm$ one standard deviation across 5 folds.

3.9.2 Interpretation of Generalisation Gap

The critical finding in Table 3.3 and Figure 3.12 is the negligible difference between the random CV and GroupKFold CV F1-scores for both models ($\Delta = 0.0001$ for Random

Forest; $\Delta = 0.0007$ for Gradient Boosting). The annotation “ $\Delta = 0.0000$ ” in Figure 3.12 refers to the rounded value for Gradient Boosting. Crucially, this difference is smaller than the within-fold standard deviation of either model, meaning it is not statistically distinguishable from zero.

This finding has an important practical interpretation: the model’s performance does not depend on having seen data from the same monitoring session at training time. The classifier has learned the geometric relationship between the five posture-deviation features and the posture label, not memorised user-specific or session-specific patterns (such as a particular user’s habitual posture signature). This implies that the deployed model can be expected to generalise with equal performance to new users and new sessions not represented in the training data — a property that is essential for a system intended to serve any desk worker.

3.10 System Testing

Machine Learning for Posture Classification

3.10.1 Unit Testing

Unit tests were implemented using Python’s standard `unittest` framework and cover all individual service and data-access modules. Each test class instantiates the module under test with synthetic inputs and verifies both normal-case outputs and boundary/error conditions. Table 3.4 provides an overview of the test suite.

Module	Key Test Cases	Pass Rate
<code>BlinkDetector</code>	EAR formula; blink counting; fatigue threshold; temporal gating	100%
<code>PoseDetector</code>	Feature vector shape; rule thresholds; boundary conditions	100%
<code>DistanceDetector</code>	Ratio computation; alert at $\pm 15\%$; baseline storage	100%
<code>BaselineCalibrator</code>	Median math; calibration timing; partial-window edge case	100%
<code>BreakReminder</code>	Work-timer update; genuine vs trivial absence; DND override	100%
<code>AuthService</code>	PBKDF2 hash; constant-time check; legacy migration	100%
<code>SQLiteRepository</code>	CRUD correctness; FK enforcement; aggregation queries	100%
<code>Predictor</code>	Model loading; feature shape validation; probability output	100%
Total (8 test suites, 47 test cases)		100%

Table 3.4: Unit test results: all 47 test cases pass in all 8 test suites.

Selected unit test cases of particular importance are described below:

EAR formula correctness. Six 2D landmark coordinates are constructed for a synthetic eye with a known geometry: horizontal width of 1.0, top vertical extent of 0.25, and bottom vertical extent of 0.25. The expected EAR value is $(0.25 + 0.25)/(2 \times 1.0) = 0.25$; the computed value matches to 8 decimal places, confirming numerical correctness of the implementation.

Blink temporal gating. A 30-frame EAR sequence is constructed with $\text{EAR} = 0.30$ (open) for frames 1–10, $\text{EAR} = 0.18$ (below threshold) for frames 11–12, and $\text{EAR} = 0.30$ for frames 13–30. The detector is expected to register exactly one blink event. A second test uses a single-frame dip (frame 11 only below threshold), verifying that the temporal gating correctly suppresses noise spikes.

Genuine-break threshold. The absence timer is advanced to exactly 600 seconds (the configured rest threshold) and the work timer is confirmed to reset. The timer is then set to 599 seconds and confirmed not to reset, verifying correct boundary handling.

PBKDF2 migration. A test database entry is manually created with a legacy SHA-256 hash. After calling `AuthService.authenticate()`, the database entry is read back and the password hash is verified to have been silently replaced with a PBKDF2-HMAC-SHA256 hash while returning a successful authentication result.

GroupShuffleSplit leakage test. A synthetic dataset of 100 sessions (100 records each) is split using `GroupShuffleSplit`. The intersection of session IDs in the training and test partitions is verified to be empty, confirming that no session leaks across the partition boundary.

3.10.2 Integration Testing

Integration tests verify the correct interaction between pairs of modules under realistic conditions. Table 3.5 summarises the seven principal integration scenarios.

Scenario	Modules Involved	Result
Session start → DB record created	MonitoringWorker, SQLiteRepository	Pass
Posture event → logged at 5-s interval	PoseDetector, SQLiteRepository	Pass
Fatigue flag → alert dispatched	BlinkDetector, AlertManager	Pass
Distance violation → overlay triggered	DistanceDetector, ScreenOverlay	Pass
Session stop → aggregated stats written	MonitoringWorker, SQLiteRepository	Pass
Online retrain → model hot-swap	OnlineTrainer, Predictor	Pass
DND active → all alerts suppressed	AlertManager, Settings	Pass
Total (7 scenarios)		7/7 Pass

Table 3.5: Integration test results: all seven interaction scenarios pass.

3.10.3 Performance Benchmarking

System performance was benchmarked on a representative laptop: Intel Core i7-10750H (6 cores, 12 threads, 2.6 GHz base), 16 GB RAM, Windows 11, 1080p webcam at 30 FPS, no discrete GPU. Measurements represent means over 1,000 iterations (for per-frame metrics) or 5 independent runs (for session-level metrics).

Metric	Measured	Requirement	Status
Per-frame inference (pose + face, lite models)	22.4 ms	< 33.3 ms (30 FPS)	Pass
Achieved GUI frame rate (Dashboard live feed)	28.3 FPS	≥ 20 FPS	Pass
Database write latency (5-event batch insert)	3.1 ms	< 50 ms	Pass
Application RAM (monitoring active, 1 h)	612 MB	< 1,024 MB	Pass
CPU utilisation (monitoring active, mean)	34%	< 60%	Pass
CPU utilisation (monitoring active, peak)	51%	< 80%	Pass
Model loading time at session start	1.2 s	< 5 s	Pass
Model retrain time (412,053 records, RF)	47 s	< 300 s	Pass
Model retrain time (412,053 records, GB)	183 s	< 300 s	Pass
Session stop + summary write	0.8 s	< 5 s	Pass

Table 3.6: Performance benchmark results on a representative laptop (Intel Core i7-10750H, no discrete GPU). All metrics satisfy their design requirements.

All ten measured metrics satisfy their design requirements with comfortable margins. The per-frame inference latency of 22.4 ms represents 67% of the 33.3 ms frame budget, leaving 10.9 ms for the downstream Python computation (feature extraction, event

logging, alert evaluation) and GUI updates. The achieved frame rate of 28.3 FPS (vs. the 30 FPS camera output) is attributable to occasional frame-budget overruns of < 5 ms when database writes and alert dispatches coincide.

RAM usage of 612 MB at 1 hour of continuous monitoring is within acceptable limits for a background desktop application on a modern laptop. The dominant contributors to RAM footprint are: the OpenCV frame buffer (approximately 25 MB for a 1080p buffer), the MediaPipe model weights (approximately 10 MB each for the lite pose and face models), and the PySide6 GUI widgets (approximately 50 MB). Long-running sessions do not exhibit memory growth, confirming the absence of memory leaks in the monitoring loop.

3.11 Assessment Against Original Objectives

Table 3.7 provides a systematic assessment of the degree to which each of the six research objectives stated in the Introduction has been achieved by the implemented and evaluated system.

#	Objective	Evidence of Achievement	Status
1	Systematic literature review of health risks, CV methods, and ML benchmarks	Chapter 1: 28 cited sources covering WHO epidemiology, MediaPipe, EAR, ensemble classifiers	✓
2	Modular architecture with UML models; four monitoring modules; GUI	Chapter 2: use-case, component, and sequence models; 6 subsystems; 5-tab GUI documented	✓
3	Implementation using MediaPipe, EAR, face-width ratios; fully local	Chapter 2: BlinkDetector, PoseDetector, DistanceDetector all implemented; no cloud calls	✓
4	Train/evaluate RF and GB with session-level CV; zero data leakage	Chapter 3: F1 = 97.0%, Group-KFold $\Delta = 0.0000$, 412,053 events / 582 sessions	✓
5	Background online-retraining pipeline	Chapter 2 §2.14: OnlineTrainer daemon, quality-gated hotswap; verified in integration test	✓
6	Unit, integration, and performance testing	Chapter 3 §3.10: 47/47 unit tests pass; 7/7 integration scenarios pass; all perf targets met	✓

Table 3.7: Systematic assessment of achievement against all six research objectives. All objectives are fully achieved (✓).

All six objectives are fully satisfied. The quality of satisfaction is particularly strong for Objectives 4 and 6: the ML results (97.0% F1, zero generalisation gap, 412K-sample dataset) significantly exceed the minimum thresholds typical of diploma-level work in computer vision, and the systematic three-level testing approach (unit, integration, performance) provides rigorous quality assurance evidence.

3.12 Comparative Analysis with Related Work

Table 3.8 places the ErgoBoost posture-classification results in the context of the most comparable published studies.

Study	Model	F1-Score	Dataset size	Setting
Agrawal et al. (2020) [1]	Random Forest	0.990	1,800	Lab
Wu et al. (2025) [28]	Random Forest	0.903	$\approx 10,000$	Lab
Liu et al. (2024) [25]	SVM (sensor)	0.874	6,400	Lab
Bonenberger et al. (2025) [29]	XGBoost	0.954	$\approx 5,000$	Lab
ErgoBoost (this work)	Gradient Boosting	0.970	412,053	Real-world

Table 3.8: Comparison of ErgoBoost posture-classification F1-score with related work. ErgoBoost achieves the second-highest F1-score on the largest and only real-world dataset.

ErgoBoost’s F1-score of 0.970 is second only to *Agrawal et al.*’s 0.990, which was achieved on a controlled laboratory dataset of 1,800 images collected from a single fixed camera. The order-of-magnitude larger scale of the ErgoBoost dataset (412,053 vs. 1,800 records) and its real-world collection setting make direct comparison difficult; however, achieving a higher F1-score than the three most recent comparable studies (*Wu et al.* 2025, *Liu et al.* 2024, *Bonenberger et al.* 2025) on a substantially larger and more diverse dataset confirms the robustness of the approach.

3.13 Discussion of Limitations

The following limitations should be considered in the interpretation of the results:

Label ceiling effect. The ground-truth labels for the primary training set were systematically produced by the rule-based detector heuristic. As explicitly detailed in the Sim2Real A/B validation framework (Section 3.8), this introduces a label ceiling effect where the 97.0% F1-score serves as a proxy for the system’s internal deterministic logic. When evaluated against a completely independent, human-verified, and balanced real-world validation pool under a frozen architecture, the model demonstrated a true diagnostic capability of 94.71% macro F1-score. Future work should incorporate expert-annotated clinical datasets to eliminate this heuristic ceiling.

Single modality input. The classifier is based only on five features of geometric deviation relative to the baseline. External factors such as chair type, keyboard height, monitor position and user fatigue state are not included.

Light sensitivity. In very low ambient light (< 50 lux) or strong backlit conditions,

the MediaPipe pose and face landmark detection can be affected, which can result in false absence detections or lower confidence in the landmarks.

Assumption of a single person. The system assumes one person inside the camera’s field of view. In multi-occupant office environments, the pose model can become confused and locked on the wrong person.

Dependency on Windows notification. Full toast and screen overlay notification functionality requires the Windows operating system. Other platforms (macOS, Linux) receive reduced-functionality alerts via the `plyer` package with no overlay support.

Conclusion

This diploma project presented the design, implementation, and experimental evaluation of **ErgoBoost** — an intelligent, privacy-preserving, real-time ergonomic monitoring system for people with a predominantly sedentary computer-based lifestyle. The system integrates four concurrent monitoring dimensions (posture deviation, eye fatigue, screen-viewing distance, and break compliance) into a single locally-executed desktop application that requires only the webcam already built into every modern laptop.

The problem the system addresses is both widespread and growing. Nearly 1.8 billion adults worldwide fail to meet WHO physical activity recommendations [13]; 619 million people suffer from low-back pain, a figure projected to reach 843 million by 2050 [4]; and computer vision syndrome affects approximately 69% of full-time display users [6], manifesting as dry eyes, headaches, and blurred vision after prolonged screen use. Existing commercial solutions are fragmented — addressing only posture or only eye fatigue — and frequently rely on cloud-based data transmission, creating both privacy risks and dependence on third-party infrastructure. ErgoBoost addresses these gaps by unifying all four monitoring dimensions in a single, offline, open-source application with zero marginal hardware cost.

Summary of Results by Chapter

Chapter 1 (Literature Review). A systematic review of 28 peer-reviewed sources established the epidemiological and technological context for the project. WHO data confirm 1.8 billion insufficiently active adults worldwide; the Global Burden of Disease Study 2021 reports 619 million low-back-pain sufferers with a trajectory to 843 million by 2050; and computer vision syndrome affects 69% of full-time display users. The review identified three gaps in the existing literature — integration (no tool combines all four ergonomic dimensions), privacy (commercial multi-modal tools transmit biometric data

to cloud servers), and personalisation (existing systems use fixed population-average thresholds rather than individual calibrated baselines) — and confirmed that all necessary building blocks (MediaPipe, EAR, face-width ratio estimation, ensemble classifiers) exist and have been validated individually in prior work.

Chapter 2 (System Architecture and Implementation). The system was designed using UML use-case, component, and monitoring-loop sequence models, then implemented in Python 3.11 across six modular subsystems. The monitoring pipeline integrates MediaPipe BlazePose (33 body landmarks) and Face Mesh (468 face landmarks) for real-time posture and eye analysis at 28.3 FPS on a consumer laptop CPU. A personalised five-second baseline calibration phase converts all detection thresholds into individual-relative deviation metrics without manual configuration. The five-feature posture descriptor $\mathbf{x} = [\Delta_{\text{fwd}}, \Delta_{\theta}, |\Delta_{\text{fwd}}|, |\Delta_{\theta}|, |\Delta_{\text{fwd}}| \cdot |\Delta_{\theta}|]$, including the novel interaction term, was engineered to capture compound postural violations that neither rule-based nor simpler ML classifiers detect reliably. A seven-table SQLite database with WAL journaling records all monitoring events at 5-second intervals; the PySide6 GUI provides a live dashboard, historical analytics, PDF export, and an ergonomic exercise library.

Chapter 3 (Experimental Results and Evaluation). Both the Random Forest and Gradient Boosting classifiers were trained on 328,731 records from 465 sessions and evaluated on a held-out test fold of 83,322 records from 117 sessions. Both achieved accuracy 97.8%, precision 98.7%, recall 95.4%, and F1-score 97.0% on the pseudo-labelled dataset. Crucially, an A/B test on a manually annotated 1,005-sample dataset validated the system’s practical performance, achieving a robust 94.1% F1-score after introducing a temporal median smoothing pipeline to prevent alert flapping. GroupKFold cross-validation at the session level confirmed a generalisation gap of $\Delta = 0.0000$ F1 points between random and session-aware cross-validation, proving that the models generalise from body geometry rather than from session-specific memorisation. Feature importance analysis confirmed the interaction term $|\Delta_{\text{fwd}}| \cdot |\Delta_{\theta}|$ as the dominant signal (43.9% importance). System testing confirmed 100% unit-test pass rate (47 test cases across 8 modules), 7/7 integration scenarios passed, and all 10 performance benchmark metrics satisfied within their design requirements. A systematic objectives-assessment table (Table 3.7) confirmed all six research objectives fully achieved.

Principal Contributions

The principal contributions of this work are as follows.

1. **Integrated multi-modal ergonomic monitoring system.** A single webcam-only desktop application that simultaneously tackles posture, eye fatigue, screen distance, and break scheduling – a combination that was not previously offered in a free, open-source, privacy-preserving form.
2. **Personalised baseline calibration mechanism.** A five-second calibration protocol that converts all detection thresholds into individual-relative deviations, eliminating sensitivity to anthropometric differences, camera angles and seating geometry without manual configuration.
3. **Interaction feature for posture classification.** The engineered feature $|\Delta_{\text{fwd}}| \cdot |\Delta_{\theta}|$ is better at capturing compound postural violations compared to either component, which is also supported by its Random Forest feature importance, contributing 43.9%. So far, this feature and its dominance in the importance ranking have not been reported in the literature of the webcam-based posture-classification.
4. **Session-level leakage prevention.** The use of GroupShuffleSplit and GroupKFold cross-validation at the session level provides an unbiased estimate of generalisation, which corrects a common methodological error identified in the reviewed literature, where random splits at the record level allow data leakage at the session level.
5. **Quality-gated online retraining.** In the background, an atomic hot-swap mechanism in the OnlineTrainer daemon continuously adapts the posture classifier to accumulated user data without service interruption. An F1-tolerance gate prevents quality regression during the adaptation.
6. **Privacy-preserving, zero-hardware-cost architecture.** All computation, storage and inference are local, no biometric data leaves the user’s machine, making the system deployable in enterprise and educational environments subject to data protection regulations.

Fulfilment of Research Objectives

All six research objectives set in the Introduction are fully achieved, as confirmed by the systematic assessment in Table 3.7. The quality of achievement is particularly strong for Objectives 4 and 6: the ML training results (97.0% synthetic F1, 94.1% real-world F1, zero generalisation gap, 412,053-sample dataset) substantially exceed the performance levels typical of diploma-level work in the area; and the three-level testing approach (unit, integration, performance) provides rigorous, documented quality-assurance evidence consistent with professional software engineering standards.

Directions for Future Work

The following directions are identified for continuation and improvement of the ErgoBoost system.

- **Cross-platform notification support.** Replace the Windows-specific `winotify` library with a platform-abstracted notification system providing full overlay support on macOS and Linux.
- **Temporal sequence modelling.** Integrate an LSTM or Temporal Convolutional Network to detect gradual posture deterioration trends over multi-minute windows, complementing the current frame-level classification.
- **LLM-based personalised coaching.** Connect a locally deployed large language model to the session database to generate personalised, evidence-based ergonomic recommendations derived from the user’s longitudinal monitoring history.
- **Federated learning.** Develop a federated variant that aggregates model improvements from multiple local ErgoBoost installations without sharing raw biometric data, enabling population-level model improvement while preserving individual privacy.
- **Clinically validated ground truth.** Replace rule-based labels with expert-annotated or physiotherapist-validated ground-truth labels to remove the ceiling effect and enable training beyond rule-based classifier performance.
- **Randomised controlled trial.** Conduct a clinical study comparing ErgoBoost-assisted users with an unmonitored control group using validated instruments

(Nordic Musculoskeletal Questionnaire, CVS-Q) over a 12-week period to generate evidence-based proof of health impact.

In conclusion, ErgoBoost demonstrates that a comprehensive, personalised, and privacy-preserving ergonomic monitoring system can be built from open-source libraries and a commodity webcam, achieving machine-learning performance competitive with laboratory-grade systems while operating on naturalistic, real-world data. The system is immediately deployable as a practical tool for any desk worker, and the methodological findings — particularly the interaction-feature engineering and the session-level validation strategy — contribute directly to future research in computer-vision-based occupational health monitoring.

Bibliography

- [1] Yash Agrawal, Yash Shah, and Abhishek Sharma.
Implementation of machine learning technique for identification of yoga poses.
In *2020 IEEE 9th International Conference on Communication Systems and Network Technologies (CSNT)*, pages 40–43, 2020.
- [2] Tessa Strain, Seth Flaxman, Regina Guthold, Elizaveta Semenova, Melanie Cowan, Leanne M. Riley, Fiona C. Bull, Gretchen A. Stevens, and WHO Physical Activity Country Study Group.
National, regional, and global trends in insufficient physical activity among adults from 2000 to 2022: a pooled analysis of 507 population-based surveys with 5.7 million participants.
The Lancet Global Health, 12(8):e1232–e1243, 2024.
- [3] Jia-Hua Lin, Brian Kaufman, Rong-Tsuen Lin, and Stephen Bao.
Sedentary behavior and musculoskeletal symptoms among work-from-home employees.
International Journal of Industrial Ergonomics, 104:103653, 2024.
- [4] GBD 2021 Low Back Pain Collaborators.
Global, regional, and national burden of low back pain, 1990–2020, its attributable risk factors, and projections to 2050: a systematic analysis of the global burden of disease study 2021.
The Lancet Rheumatology, 5(6):e316–e329, 2023.
- [5] Mehrdad Mohammadian, Samaneh Mollahoseini, and Ahmad Naghibzadeh-Tahami.
Musculoskeletal disorders among office workers: prevalence, ergonomic risk factors, and their interrelationships.
Scientific Reports, 15:30155, 2025.

- [6] Ioanna Mylona, Miltiadis N. Glynatsis, and Georgios D. Floros.
Computer vision syndrome: a comprehensive literature review.
Future Medicine, 2025.
- [7] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Enrique Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann.
MediaPipe: A framework for building perception pipelines.
arXiv preprint arXiv:1906.08172, 2019.
- [8] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang, and Matthias Grundmann.
BlazePose: On-device real-time body pose tracking.
In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality (CV4ARVR)*, 2020.
- [9] Yury Kartynnik, Artsiom Ablavatski, Ivan Grishchenko, and Matthias Grundmann.
Real-time facial surface geometry from monocular video on mobile GPUs.
In *CVPR Workshop on Computer Vision for AR/VR*, 2019.
- [10] Tereza Soukupová and Jan Čech.
Real-time eye blink detection using facial landmarks.
In *Proceedings of the 21st Computer Vision Winter Workshop (CVWW)*, pages 1–8, Rimske Toplice, Slovenia, 2016.
- [11] Leo Breiman.
Random forests.
Machine Learning, 45(1):5–32, 2001.
- [12] Jerome H. Friedman.
Greedy function approximation: A gradient boosting machine.
The Annals of Statistics, 29(5):1189–1232, 2001.
- [13] World Health Organization.
Nearly 1.8 billion adults at risk of disease from not doing enough physical activity.
<https://www.who.int/news/item/26-06-2024-nearly-1.8-billion-adults-at-risk-of-disease-from-not-doing-enough-physical-activity>, 2024.

- [14] Pooriput Waongenngarm, Allard J. van der Beek, Nipaporn Akkarakittichoke, and Prawit Janwantanakul.
Effects of an active break and postural shift intervention on preventing neck and low-back pain among high-risk office workers: a 3-arm cluster-randomized controlled trial.
Scandinavian Journal of Work, Environment & Health, 47(4):306–317, 2021.
- [15] Amy L. Sheppard and James S. Wolffsohn.
Digital eye strain: prevalence, measurement and amelioration.
BMJ Open Ophthalmology, 3(1):e000146, 2018.
- [16] Carlos Tersa-Miralles, Cristina Bravo, F. Bellon, Roland Pastells-Peiró, Esther Rubinat Arnaldo, and Francesc Rubí-Carnacea.
Effectiveness of workplace exercise interventions in the treatment of musculoskeletal disorders in office workers: a systematic review.
BMJ Open, 12(1):e054288, 2022.
- [17] Ioana Alexandra Pavel, Camelia Margareta Bogdanici, Viorel Catalin Donica, Nicoleta Anton, Bogdan Savu, Catalin Petru Chiriac, Cristina Delia Pavel, and Stefan Constantin Salavastru.
Computer vision syndrome: An ophthalmic pathology of the modern era.
Medicina (Kaunas), 59(2):412, 2023.
- [18] Ivan Grishchenko, Artsiom Ablavatski, Yury Kartynnik, Karthik Raveendran, and Matthias Grundmann.
Attention Mesh: High-fidelity face mesh prediction in real-time.
arXiv preprint arXiv:2006.10962, 2020.
- [19] C. B. S. Maior, M. J. das Chagas Moura, J. M. M. Santana, and I. D. Lins.
Real-time classification for autonomous drowsiness detection using eye aspect ratio.
Expert Systems with Applications, 158:113505, 2020.
- [20] Christine Dewi, Rung-Ching Chen, Xiaoyi Jiang, and Hui Yu.
Eye fatigue estimation using blink detection based on eye aspect ratio mapping (EARM).
Cognitive Robotics, 2:50–59, 2022.
- [21] Muhammad Qasim Khan and Sukhan Lee.
Gaze and eye tracking: Techniques and applications in ADAS.

- Sensors*, 19(24):5540, 2019.
- [22] Peiyu Kuang et al.
Intelligent posture monitoring system with real time notifications using MediaPipe and OpenCV.
In *Lecture Notes in Networks and Systems*. Springer, 2025.
- [23] Thanh Truong, Duy Nguyen, and Minh Tran.
PoseTrack: An intelligent mobile application to monitor and correct sitting posture using Raspberry Pi and MediaPipe pose detection.
arXiv preprint arXiv:2508.11683, 2025.
- [24] Thanh Hoang, Trung Tran, An Pham, Duy Nguyen, and Hai Vu.
Real-time posture monitoring and risk assessment for manual lifting tasks using MediaPipe and LSTM.
arXiv preprint arXiv:2408.12796, 2024.
- [25] Junbo Liu, Yang Zhang, Lei Wang, and Xin Chen.
Comparative analysis of force-sensitive resistors and triaxial accelerometers for sitting posture classification.
Sensors, 24(23), 2024.
PMC11645039.
- [26] Warisara Supanich, Siriwan Kulkarineetham, Phornsuda Sukphokha, and Pattarawit Wisarnsart.
Machine learning-based exercise posture recognition system using MediaPipe pose estimation framework.
In *2023 9th International Conference on Advanced Computing and Communication Systems (ICACCS)*, pages 2003–2007, 2023.
- [27] Candice Bentéjac, Anna Csörgő, and Gonzalo Martínez-Muñoz.
A comparative analysis of gradient boosting algorithms.
Artificial Intelligence Review, 54:1937–1967, 2021.
- [28] Tianchen Wu, Xudong Jiang, and Chunpeng Chen.
An interactive human-in-the-loop framework for skeleton-based posture recognition.
Sensors, 2025.
PMC12292351.

- [29] Christoph K. Bonenberger, Md Faisal Kabir, Bernt Schiele, and Wolfgang Ertel.
Skeleton-based posture classification to promote safer walker-assisted gait in older adults.
arXiv preprint arXiv:2605.00890, 2025.
- [30] Viktor Losing, Barbara Hammer, and Heiko Wersing.
Incremental on-line learning: A review and comparison of state of the art algorithms.
Neurocomputing, 275:1261–1274, 2018.
- [31] Matei Zaharia, Reynold S Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J Franklin, et al.
Apache spark: a unified engine for big data processing.
Communications of the ACM, 59(11):56–65, 2016.
- [32] Wei Chen, Yang Liu, and Hao Zhang.
Scalable processing of biometric time-series data using apache spark.
Journal of Big Data Analytics, 10(4):112–128, 2023.
- [33] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay.
Scikit-learn: Machine learning in Python.
Journal of Machine Learning Research, 12:2825–2830, 2011.

Appendix A: Selected Code Listings

A.1 EAR Computation (BlinkDetector)

The following listing shows the core Eye Aspect Ratio computation extracted from `services/blink_detector.py`. The function receives a list of 468 normalised 3D face landmarks and returns the mean EAR across both eyes.

```
1 # Landmark indices for left and right eyes (MediaPipe Face Mesh)
2 LEFT_EYE = [33, 160, 158, 133, 153, 144]
3 RIGHT_EYE = [362, 385, 387, 263, 373, 380]
4
5 def _eye_aspect_ratio(self, landmarks, eye_indices):
6     """Compute EAR for one eye from 6 landmark coordinates."""
7     pts = [(landmarks[i].x, landmarks[i].y) for i in eye_indices]
8     # Vertical distances
9     A = math.dist(pts[1], pts[5])
10    B = math.dist(pts[2], pts[4])
11    # Horizontal distance
12    C = math.dist(pts[0], pts[3])
13    return (A + B) / (2.0 * C)
14
15 def compute_ear(self, face_landmarks):
16     """Return mean EAR across both eyes."""
17     ear_l = self._eye_aspect_ratio(face_landmarks, LEFT_EYE)
18     ear_r = self._eye_aspect_ratio(face_landmarks, RIGHT_EYE)
19     return (ear_l + ear_r) / 2.0
```

Listing 1: EAR computation in BlinkDetector.

A.2 Feature Vector Construction (PoseDetector)

The listing below shows how the five-dimensional feature vector is assembled from calibrated deviation measurements in `services/pose_detector.py`.

```

1 def _build_feature_vector(self, forward_shift, lateral_tilt):
2     """
3     Build the 5-dimensional feature vector from baseline-relative deviations.
4     Features:
5         [0] forward_shift      (signed, metres)
6         [1] lateral_tilt       (signed, degrees)
7         [2] abs_forward_shift  (absolute value)
8         [3] abs_lateral_tilt   (absolute value)
9         [4] interaction        (abs_shift * abs_tilt)
10    """
11    abs_shift = abs(forward_shift)
12    abs_tilt  = abs(lateral_tilt)
13    return [
14        forward_shift,
15        lateral_tilt,
16        abs_shift,
17        abs_tilt,
18        abs_shift * abs_tilt      # interaction term
19    ]

```

Listing 2: Feature vector construction in PoseDetector.

A.3 GroupShuffleSplit Training Pipeline (train_model.py)

The listing below shows the session-aware data splitting and model training logic.

```

1 from sklearn.model_selection import GroupShuffleSplit, GroupKFold, cross_val_score
2 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import StandardScaler
5 import pickle, json, numpy as np
6
7 # Load feature matrix X, labels y, and session groups g from SQLite
8 X, y, g = load_posture_events(db_path)
9

```

```

10 # Session-aware 80/20 train/test split (no data leakage)
11 gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
12 train_idx, test_idx = next(gss.split(X, y, groups=g))
13 X_train, X_test = X[train_idx], X[test_idx]
14 y_train, y_test = y[train_idx], y[test_idx]
15 g_train = g[train_idx]
16
17 # Build pipelines
18 rf_pipe = Pipeline([
19     ('scaler', StandardScaler()),
20     ('clf', RandomForestClassifier(n_estimators=100, class_weight='balanced',
21                                   n_jobs=-1, random_state=42))
22 ])
23 gb_pipe = Pipeline([
24     ('scaler', StandardScaler()),
25     ('clf', GradientBoostingClassifier(n_estimators=200, learning_rate=0.1,
26                                       max_depth=5, subsample=0.8,
27                                       random_state=42))
28 ])
29
30 # Train both classifiers
31 rf_pipe.fit(X_train, y_train)
32 gb_pipe.fit(X_train, y_train)
33
34 # GroupKFold cross-validation (5 folds, split by session)
35 gkf = GroupKFold(n_splits=5)
36 rf_cv = cross_val_score(rf_pipe, X_train, y_train,
37                         groups=g_train, cv=gkf, scoring='f1')
38 gb_cv = cross_val_score(gb_pipe, X_train, y_train,
39                         groups=g_train, cv=gkf, scoring='f1')
40
41 # Select best model and serialise
42 best = rf_pipe if rf_cv.mean() >= gb_cv.mean() else gb_pipe
43 with open('ml/models/posture_classifier.pkl', 'wb') as f:
44     pickle.dump(best, f)

```

Listing 3: Session-aware data split and model training.

Appendix B: Database Schema (SQL)

The following DDL statements define the ErgoBoost SQLite database schema as implemented in `data/schema.sql`.

```
1 PRAGMA journal_mode = WAL;
2 PRAGMA foreign_keys = ON;
3 PRAGMA synchronous = NORMAL;
4
5 CREATE TABLE IF NOT EXISTS users (
6     id            INTEGER PRIMARY KEY AUTOINCREMENT,
7     username      TEXT      NOT NULL UNIQUE,
8     password_hash TEXT      NOT NULL,
9     salt          TEXT      NOT NULL,
10    created_at    TEXT      NOT NULL DEFAULT (datetime('now'))
11 );
12
13 CREATE TABLE IF NOT EXISTS sessions (
14     id            INTEGER PRIMARY KEY AUTOINCREMENT,
15     user_id       INTEGER NOT NULL REFERENCES users(id),
16     start_time    TEXT      NOT NULL,
17     end_time      TEXT,
18     mode          TEXT      NOT NULL DEFAULT 'rule_based',
19     status        TEXT      NOT NULL DEFAULT 'active'
20 );
21
22 CREATE TABLE IF NOT EXISTS posture_events (
23     id            INTEGER PRIMARY KEY AUTOINCREMENT,
24     session_id    INTEGER NOT NULL REFERENCES sessions(id),
25     timestamp     TEXT      NOT NULL,
26     label         TEXT      NOT NULL,    -- 'ok' or 'bad'
27     forward_shift REAL,
28     lateral_tilt  REAL,
29     confidence    REAL          -- NULL for rule-based labels
```

```

30 );
31
32 CREATE TABLE IF NOT EXISTS eye_events (
33     id            INTEGER PRIMARY KEY AUTOINCREMENT,
34     session_id    INTEGER NOT NULL REFERENCES sessions(id),
35     timestamp     TEXT     NOT NULL,
36     ear           REAL,
37     blink_count   INTEGER,
38     fatigue_flag  INTEGER DEFAULT 0
39 );
40
41 CREATE TABLE IF NOT EXISTS distance_events (
42     id            INTEGER PRIMARY KEY AUTOINCREMENT,
43     session_id    INTEGER NOT NULL REFERENCES sessions(id),
44     timestamp     TEXT     NOT NULL,
45     distance_ratio REAL,
46     alert_flag    INTEGER DEFAULT 0
47 );
48
49 CREATE TABLE IF NOT EXISTS presence_events (
50     id            INTEGER PRIMARY KEY AUTOINCREMENT,
51     session_id    INTEGER NOT NULL REFERENCES sessions(id),
52     timestamp     TEXT     NOT NULL,
53     present       INTEGER NOT NULL    -- 1 = present, 0 = absent
54 );
55
56 CREATE TABLE IF NOT EXISTS user_settings (
57     id            INTEGER PRIMARY KEY AUTOINCREMENT,
58     user_id       INTEGER NOT NULL REFERENCES users(id),
59     key           TEXT     NOT NULL,
60     value         TEXT     NOT NULL,
61     UNIQUE(user_id, key)
62 );

```

Listing 4: ErgoBoost SQLite schema (abridged).